

Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering

Hamed Jelodar ^{a,*}, Samita Bai ^a, Parisa Hamed ^a, Hesamodin Mohammadian ^a,
Roozbeh Razavi-Far ^a, Ali Ghorbani ^a

^a Canadian Institute for Cybersecurity, Faculty of Computer Science, University of New Brunswick, 46 Dineen Dr #134, Fredericton, NB E3B 9W4, Canada

ARTICLE INFO

Keywords:

Large language model
Software security
Source code analysis
Large concept model
Knowledge-Enhanced pretrained
Reverse engineering

ABSTRACT

Large Language Models (LLMs) have recently emerged as powerful tools in cybersecurity, offering advanced capabilities in malware detection, generation, and real-time monitoring. Numerous studies have explored their application in cybersecurity, demonstrating their effectiveness in identifying novel malware variants, analyzing malicious code structures, and enhancing automated threat analysis. Several transformer-based architectures and LLM-driven models have been proposed to improve malware analysis, leveraging semantic and structural insights to recognize malicious intent more accurately. This study presents a comprehensive review of LLM-based approaches in malware code analysis, summarizing recent advancements, trends, and methodologies. We examine notable scholarly works to map the research landscape, identify key challenges, and highlight emerging innovations in LLM-driven cybersecurity. Additionally, we emphasize the role of static analysis in malware detection, introduce notable datasets and specialized LLM models, and discuss essential datasets supporting automated malware research. This study serves as a valuable resource for researchers and cybersecurity professionals, offering insights into LLM-powered malware detection and defense strategies while outlining future directions for strengthening cybersecurity resilience.

1. Introduction

Malicious code detection is a critical aspect of cybersecurity, as it helps protect computer systems and networks from various types of attacks. With malware's increasing sophistication and complexity, traditional detection methods often struggle to keep pace. The advent of large language models (LLMs) has opened up new possibilities for enhancing malicious code detection by leveraging the power of artificial intelligence (AI) and natural language processing (NLP) [1]. There are interesting papers based on transformers and LLMs that are applied in various fields, including Malicious code detection and PE files [2–4], code classification [5,6], code monitoring and inspection [7–10], and code generation [7,8,11].

For example, in [12], the authors focused on evaluating LLMs for zero-shot learning in detecting malicious code, comparing code-specific models (Code-llama [13]) with general-purpose models (Llama3 [14], GPT-3.5 [15], and Claude [16]). The study emphasized the effectiveness of prompts highlighting experience and maliciousness indicators over

role-playing and noted that repeated questioning reduced LLM confidence. Moreover, in [11] the authors explored the risks of LLMs being exploited for malicious code generation, such as advanced phishing malware. They created MCGTEST, a dataset with 406 tasks, and developed MCGMARK, a robust watermarking method for tracing LLM-generated code. This approach, validated on the DeepSeek-Coder model [17], achieved a high embedding success rate while maintaining the code quality.

In [18], the authors focused on developing LAMD, a framework for Android malware detection and classification using LLMs. Their primary goal was to improve detection accuracy by analyzing malware behavior at multiple levels, ranging from low-level instructions to high-level semantics. This approach allowed for context-driven predictions and explanations, enhancing the understanding of the malware's operation. The researchers utilized JADX3 [19] to decompile the malware, concatenating all the pseudo source codes for input. This step was essential for ensuring comprehensive analysis. In addition to GPT-4o-mini, they selected Gemini 1.5 Pro as another comparison model, praised for its

* Corresponding author.

E-mail addresses: h.jelodar@unb.ca (H. Jelodar), samita.bai@unb.ca (S. Bai), parisa.hamed@unb.ca (P. Hamed), h.mohammadian@unb.ca (H. Mohammadian), roozbeh.razavi-far@unb.ca (R. Razavi-Far), ghorbani@unb.ca (A. Ghorbani).

<https://doi.org/10.1016/j.jisa.2026.104390>

Available online 10 February 2026

2214-2126/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

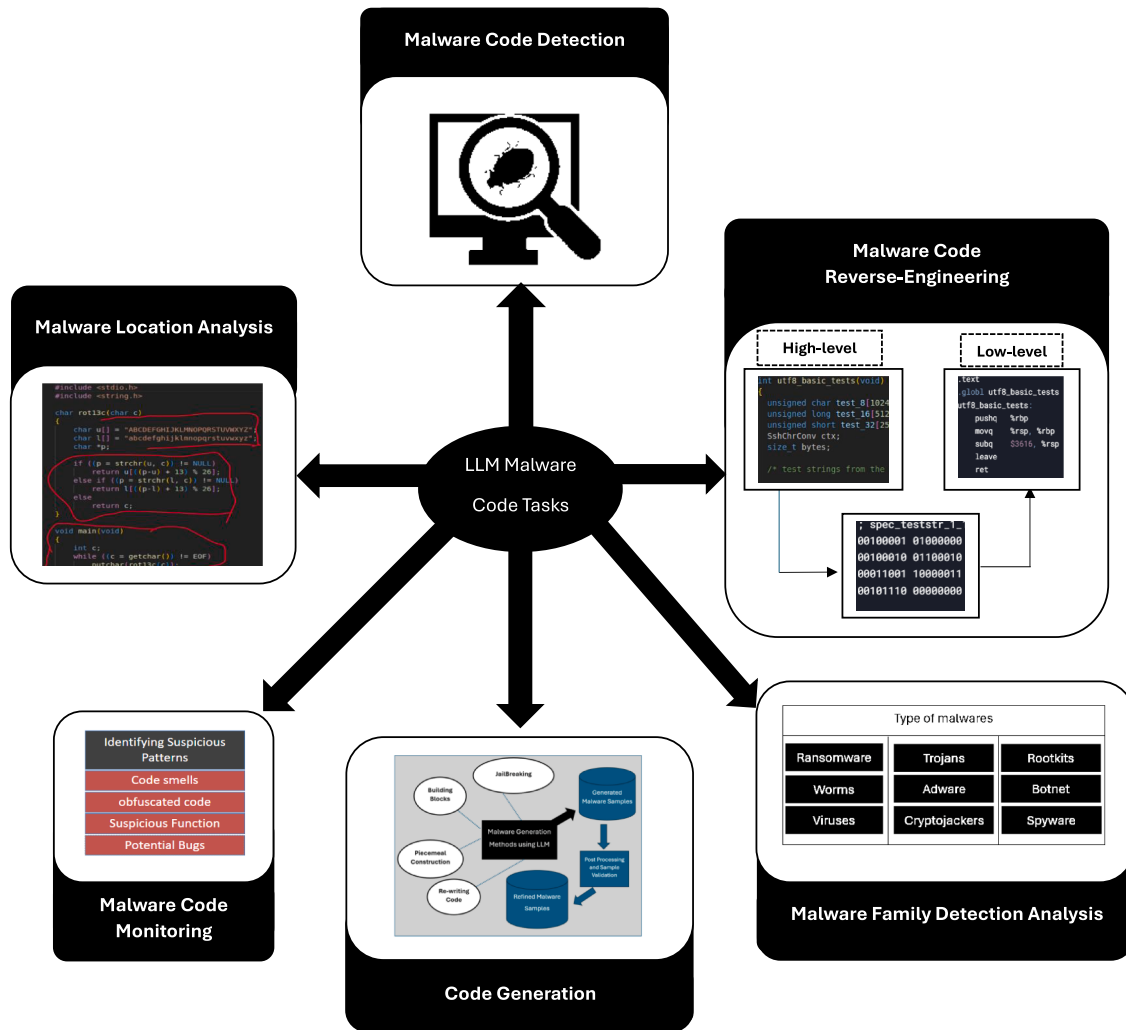


Fig. 1. An overview of various malware code aspects explored in this research.

ability to handle the longest context windows and its proven malware detection capabilities, making it a strong contender for effective malware classification.

Recent research emphasizes the remarkable potential of LLMs in malware code analysis, building on their demonstrated success in malware code understanding and analysis. LLM models leverage advanced natural language processing techniques to interpret and analyze code patterns, enabling enhanced detection of vulnerabilities and malicious behaviors. LLMs can identify subtle nuances in malware coding techniques by utilizing pre-trained knowledge from vast datasets, facilitating improved threat detection and remediation strategies. As research in this area progresses, the integration of LLMs into cybersecurity frameworks promises to revolutionize how analysts approach malware analysis and response. Generally, in this paper, we aim to explore recent papers on how LLMs analyze the semantics and logic of source code to identify malicious behaviors. We highlight their capabilities in recognizing malware patterns and discerning logical flows indicative of malicious intent. We aim to provide an overview of advancements in LLM applications for malware detection and future research directions in cybersecurity.

Research Motivation and Contributions: Given the proven effectiveness of LLMs in various analytical domains, particularly natural

language processing, and cybersecurity, we are motivated to explore their potential applications in malware code analysis. This study aims to examine how LLMs can be leveraged for detecting, analyzing, and understanding malware code through different models, datasets, and techniques. As illustrated in Fig. 1, we investigate multiple aspects of malware code analysis using LLMs. The primary objective of this work is to provide a comprehensive overview of LLM-based approaches in malware code analysis. In summary, this paper makes four key contributions:

- A review of recent works on LLM applications in malware code analysis is presented, synthesizing current research findings and trends in the field.
- The study examines how LLMs interpret the semantics and structure of source code in order to effectively identify malicious behaviors.
- The study provides a **unified technical perspective** on how LLMs interpret the semantics and structural properties of malicious source code and binaries to enable automated detection, behavioral reasoning, and code understanding.
- A quantitative comparative analysis of recent LLM-based malware analysis approaches is introduced through a unified performance

table, enabling direct cross-task comparison across malware detection, phishing analysis, behavioral replication, and family classification.

- The work further identifies key research gaps, future directions, and publicly available datasets that can facilitate the next generation of LLM-based malware analysis systems.

Review and Assess Pertinent Research: This study considered a comprehensive methodology for identifying pertinent research, encompassing the selection of data sources, search criteria, and the development of classification frameworks. The analysis is built upon an in-depth review of academic literature focused on malware detection in software code, with particular attention given to the most recent and widely cited papers that represent the forefront of research in the field. By critically examining these studies, we aim to uncover emerging trends, methodologies, and techniques that are shaping the evolution of malware detection, providing a snapshot of the latest advancements in cybersecurity. Furthermore, this study extends the discussion to the application of LLMs in the domain of malware code analysis, highlighting how these models can enhance detection, classification, and understanding of malicious code. Through this exploration, we aim to identify key opportunities for integrating LLMs into existing malware detection frameworks, offering insights into their potential to revolutionize malware analysis.

The rest of the paper is organized as follows. In Sections 2 and 3, we provide background on the core concepts of LLMs and related works. In Sections 4 to 9, we investigate and explore malware code analysis using LLMs, covering detection, generation, monitoring, reverse engineering, and family analysis. In Section 10, we discuss the most highlighted, practical, and recent LLM models with great potential for malware code analysis. Additionally, we examine extended datasets and LLMs for generating datasets for software code analysis tasks related to malware.

2. Background

To improve the understanding and readability of concepts in this paper, this section provides background on key concepts related to methodologies, malware scenarios, and malware-code analysis tasks using LLMs.

Reverse Engineering and Binary Analysis: Reverse engineering and binary analysis are essential for dissecting and understanding malware [20]. Since most malicious programs are distributed in compiled form without source code, security researchers rely on reverse engineering techniques to uncover their functionality. By analyzing binaries using tools such as IDA Pro- [21] and Ghidra [22], one can reconstruct the malware's logic and identify malicious functions.

Analyzing Malicious Software: With the increasing volume and complexity of malware, analyzing malicious software has become a critical challenge. Malware analysis involves examining malicious code to understand its purpose, behavior, and impact on infected systems [23]. This process can be carried out using static or dynamic analysis methods, both of which play a crucial role in classifying malware families and enhancing cybersecurity defenses.

2.1. Core concepts of LLMs

LLMs are a big step forward in AI, improving language understanding and generation while enhancing capabilities in multiple fields [24]. LLMs operate on advanced deep learning frameworks, primarily utilizing transformer architectures and self-attention mechanisms. Their training relies on extensive datasets drawn from a wide range of textual sources, such as web content, published literature, and open-source codebases.

- **Primally LLMs:** These LLMs are designed to handle a wide range of tasks rather than focusing on a single area. Currently, GPT and DeepSeek are popular examples of such models, excelling in logical

reasoning, solving math problems, and generating creative writing. For example, in malware disassembly [25], LLMs can be useful for analyzing malware by summarizing disassembled code and providing human-readable explanations of complex functions.

- **Prompt engineering:** Prompt engineering is a technique used to guide LLMs in generating accurate and relevant responses. It involves designing and refining prompts to help LLMs produce the desired outputs effectively [26]. The structure and quality of prompts are critical in ensuring that the model generates accurate and relevant responses. Zero-shot and few-shot prompts are the most practical techniques. For example, a well-designed zero-shot prompt could ask an LLM to analyze obfuscated malware code and suggest deobfuscation techniques.
- **Training LLMs from Scratch:** Pre-training is a crucial step in developing LLMs, involving the exposure of models to vast amounts of unlabeled text data [27]. During this phase, the model learns grammar, context, and semantic relationships without specific task supervision. This foundational training enables the model to generalize across a wide range of applications before being fine-tuned for specialized tasks. However, pre-training LLMs on malware datasets, including malware repositories and threat intelligence feeds, significantly enhances their ability to assist in malware analysis [28].
- **Adapting Models for Specific Tasks:** Fine-tuning involves further training an LLM on domain-specific data to optimize its performance for particular applications. Unlike pre-training, which provides general language understanding, fine-tuning tailors the model to specialized tasks [24]. By training models on labeled datasets containing malware behaviors, code structures, and security reports, we can create specialized AI tools to detect, classify, and analyze threats [29].
Fine-tuning can be performed using supervised learning, reinforcement learning, or human feedback-based techniques. It allows organizations to customize AI models while reducing the computational burden of training from scratch. However, fine-tuning also introduces risks, such as overfitting or reinforcing biases in training data [30].
- **Foundation Models and Architectures:** Pre-trained models and transformer architectures form the backbone of modern LLMs. Transformers, introduced in the seminal paper "Attention Is All You Need," revolutionized NLP by enabling efficient parallel processing and long-range contextual understanding [31]. These architectures use self-attention mechanisms to weigh different parts of input text, allowing for more coherent and context-aware responses.

3. Related work

LLMs have numerous applications in malware analysis. Many studies have been conducted utilizing LLM-based approaches across various domains, such as cybersecurity, threat detection, reverse engineering, and digital forensics. Some research has specifically focused on surveying LLM applications in malware analysis [32–34].

Several recent surveys have examined the role of large language models (LLMs) in cybersecurity and malware analysis [35,36]. Xu et al. [35] provide a broad systematic review of LLM applications across diverse cybersecurity domains, including threat intelligence, intrusion detection, vulnerability analysis, and cyber defense automation. Similarly, Al-Karaki et al. [36] focus on LLMs for malware detection and propose a conceptual framework along with potential countermeasure strategies.

While these surveys offer valuable high-level insights, they primarily emphasize *text-based threat intelligence*, alert analysis, and generic malware classification pipelines. In contrast, the present study adopts a **code-centric perspective**, emphasizing how LLMs operate directly on malicious *source code and binaries*. Unlike prior surveys, this work provides an integrated technical treatment of: (i) malware code detection, (ii) malware behavior monitoring, (iii) reverse engineering and deobfuscation, and (iv) malware family and semantic code analysis.

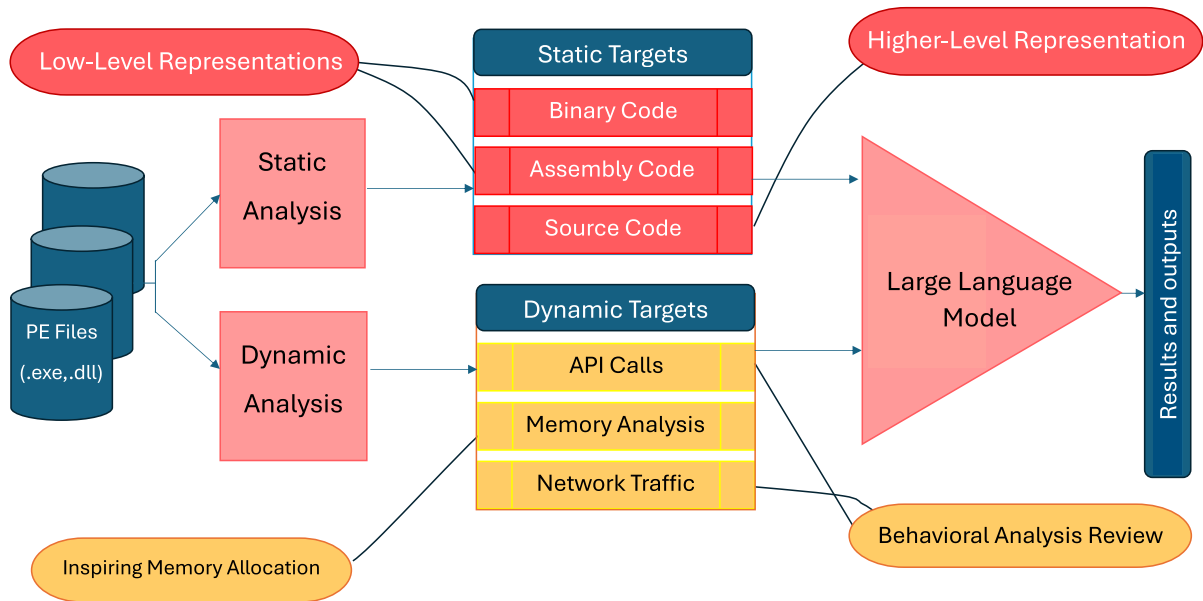


Fig. 2. A general view of malware analysis based on static and dynamic ideas for feeding LLM.

This finer task granularity enables a deeper understanding of how LLMs interact with low-level program representations, instruction semantics, control flow, API usage, and binary artifacts-capabilities that are not examined in detail by earlier survey efforts. As a result, this review complements existing literature by focusing specifically on the *program-analysis interface of LLMs*, rather than on general cybersecurity analytics alone.

3.1. Malware analysis: LLMs in static & dynamic

This section examines the application of LLMs in malware code analysis, as illustrated in Fig. 2, emphasizing their role in secure coding, malicious code detection, and automated repair generation [37–40]. Broadly, malware analysis using LLMs can be categorized into two main approaches: (1) Static Analysis and (2) Dynamic Analysis. These approaches provide a comprehensive framework for malware analysis powered by LLMs.

3.2. Static approaches

Static malware analysis methods examine file hashes or binary code without execution to identify patterns that may disrupt computer performance [41].

3.2.1. Static approaches using LLM

Different research efforts have explored the use of LLMs based on static malware analysis. For example, the authors in [42], focused on examining the application of LLMs to enhance static malware analysis, demonstrating high accuracy in describing malware functions. The authors employed a method involving static analysis workflows and utilized the OpenAI Codex model. They used a dataset comprising various malware samples to validate their findings [41]. Also, other researchers [43], addressed the increasing threat of cyber attacks by proposing automated machine learning methods for detecting malicious code using advanced deep learning models. The authors employ stacked bidirectional long short-term memory (Stacked BiLSTM) and generative pre-trained transformer (GPT-2) models, utilizing assembly instructions from the .text sections of both malicious and benign PE files.

3.2.2. Iterative malware summarization

Some prior works have examined LLM models for malware summarization. For example, in [37] the authors focused on addressing the challenges of generating accurate, complete, and user-friendly malware summaries by introducing MALSIGHT, a framework designed to improve malware behavior descriptions. They used and fine-tuned the CodeT5+ model [44] to enable transfer learning from source code summarization to decompiled pseudocode summarization. Their approach involved creating the first malware summary datasets, MalS and MalP, using LLM with manual refinement and training the specialized CodeT5+ model to iteratively summarize malware functions. Furthermore, the researchers in [45] aimed to address the challenges of complex and time-consuming reverse engineering in Android malware analysis by introducing MalParse, a framework that leverages OpenAI's GPT-4o-mini model for semantic analysis and categorization. Their approach enhances malware analysis by enabling automated, hierarchical code summarization at the function, class, and package levels, while also improving the efficiency of identifying malicious components.

3.3. Dynamic approaches

Dynamic analysis differs from static analysis by focusing on identifying malicious patterns during execution rather than analyzing binary code. Unlike static methods, which are susceptible to evasion techniques, dynamic analysis does not require code disassembly [46]. Its primary objective is to detect malicious activity in real-time while ensuring the security of the analysis platform.

3.3.1. Dynamic approaches using LLM on API calls

A handful of studies have investigated the dynamic approach and LLMs. For example, in [46], the authors focused on improving malware detection by addressing the limitations of traditional dynamic analysis, which often struggled with obfuscated or unknown API calls. They used GPT-4 to generate explanatory text for each API call and embedded these explanations using BERT to create detailed representations of API sequences. These representations were then processed by a CNN-based detection model to identify malicious behavior. Also, in [46] the authors introduced Nebula, a Transformer-based architecture for malware detection and classification, addressing the limitations of traditional dynamic

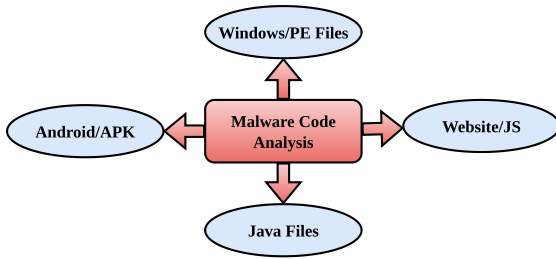


Fig. 3. A general view of applications of malicious code analysis based on different environments.

analysis, which often relied on runtime APIs while neglecting heterogeneous data like network and file operations. Nebula processes diverse behavioral data from dynamic log reports using tokenization, filtering, normalization, and encoding components.

4. LLM For malicious code detection

Recently, LLMs have provided a powerful solution for detecting both malicious and benign code by identifying patterns or malware family types. We can also see a relationship between malware code detection with different LLM methods in Fig. 4. [36]. Few works have explored this method are summarized in Table 1. Our study of previous research indicates that LLMs for malware code detection, can be applied to four environments or data formats, as shown in Fig. 3.

Additionally, as illustrated in Fig. 4, the diagram summarizes how different malware code analysis tasks are supported by various LLM and NLP methods. The central block on the left represents the overall setting of *Malware Code Analysis with LLM*, which branches into six task-specific categories shown in the colored rectangular boxes: malware code detection (orange), malware-family code analysis (dark green), malware-location and code analysis (purple), malware code generation (blue), malware code monitoring (light green), and malware code reverse engineering (grey). The references listed inside each box (e.g., [4], [23], [46]) indicate representative studies for that task. On the right, three violet circles denote the main methodological families considered in this survey: zero/few-shot prompt engineering, fine-tuning LLMs, and lightweight NLP models. Color-coded arrows connect each task box to one or more method circles, using the same color as the task box, to show which LLM/NLP strategies are predominantly employed for that category. For example, the orange arrows highlight works that use prompt engineering or fine-tuning for malware code detection, whereas the green and blue arrows illustrate that family analysis, code generation, and monitoring often combine multiple LLM paradigms. This visualization emphasizes that different malware analysis tasks draw on overlapping, but not identical, LLM and NLP techniques.

4.1. Applications and environments

Previous studies show that malware detection using static analysis and LLMs depends on the environment. This includes platforms like Android, Windows, websites, and Java. Each environment has unique features that affect detection methods. Understanding these differences is important for effective detection.

4.1.1. Static malware analysis for android

LLMs have emerged as powerful tools for malware detection and analysis, especially in Android applications [51,53]. For example; in [28], the authors stated that cyber threats and malware attacks have increased, posing significant risks to individuals and businesses. They emphasized the need for automated machine learning techniques to proactively defend against malware. To address this, they proposed MalBERT,

a BERT-based Transformer model for static malware analysis. Their approach involved analyzing the source code of Android applications, extracting preprocessed features, and classifying malware into different categories.

4.1.2. Static analysis of malware in java

There is some impressive works that only focused on Java Malware files. For example, in [33], the authors utilized the Mixtral LLM model [59] to identify malicious code in Java source files. They also leveraged NEO4J GenAI applications to analyze code structure, detect patterns, and uncover potential security threats. This approach enabled a more comprehensive understanding of malware behavior, improving detection accuracy and threat analysis.

4.1.3. Static malware analysis of websites

Website malware refers to malicious software or harmful code that infects websites with the intention of damaging or exploiting the site or its visitors. This can include a variety of different attack types, including viruses, worms, Trojans, and scripts that steal data or inject unwanted content into a site. For example in [60], the authors focused on examines the security vulnerabilities introduced when LLMs, like GPT-4, are used to generate PHP code in web development. This analysis highlights the potential for exploitation, similar to how malware could exploit weak or improperly generated code in web environments. The research emphasizes the necessity of applying robust security measures when leveraging LLMs in web applications to prevent malicious activities.

4.1.4. Static analysis of windows malware files

There have been a few notable contributions to show applications of LLM for Malware analysis on PE files. For example in [43], The authors focused on developing deep learning models, specifically Stacked BiLSTM and GPT-2, for detecting malicious code. They utilized GPT-2 models trained on assembly instructions extracted from both malicious and benign PE files. Their dataset included Win32 PE files from Windows operating systems, Commando VM v-2.0, and malicious x86 executables sourced from the s0rel-20m database and the VirusShare website. In the next section, a detailed discussion is provided for malware detection strategies.

4.2. Malware-Code detection strategies

Malware detection has advanced with LLMs, particularly through fine-tuning pre-trained models on labeled malicious code. This method improves detection, especially for obfuscated or unseen code, by helping the model recognize complex malware patterns and domain-specific features.

4.2.1. Fine-Tuning LLMs for malware code detection

Fine-tuning LLMs for malware code analysis involves tailoring a pre-trained model to specifically analyze malicious software code, helping to identify hidden threats within binaries, scripts, or other executable forms of malware [37]. By training the LLM on a dataset that includes labeled samples of both benign and malicious code, the model learns to detect unusual patterns, code obfuscation techniques, and signature-less malware behaviors. Fine-tuning enables the model to better understand programming languages, malicious coding structures, and techniques such as polymorphism or encryption, which are often used to evade traditional detection methods. This allows for more effective identification of new and evolving malware strains.

Integrating fine-tuned LLMs with malware code analysis can significantly speed up the process of reverse engineering and threat detection [24]. Additionally, the LLM can offer context-driven insights or recommendations to help interpret obscure parts of the code, such as obfuscated functions or encrypted payloads. Over time, as more malware samples are analyzed, the fine-tuned LLM can evolve to recognize emerging techniques and improve the accuracy of predictions, contributing to a proactive defense against increasingly sophisticated malware attacks.

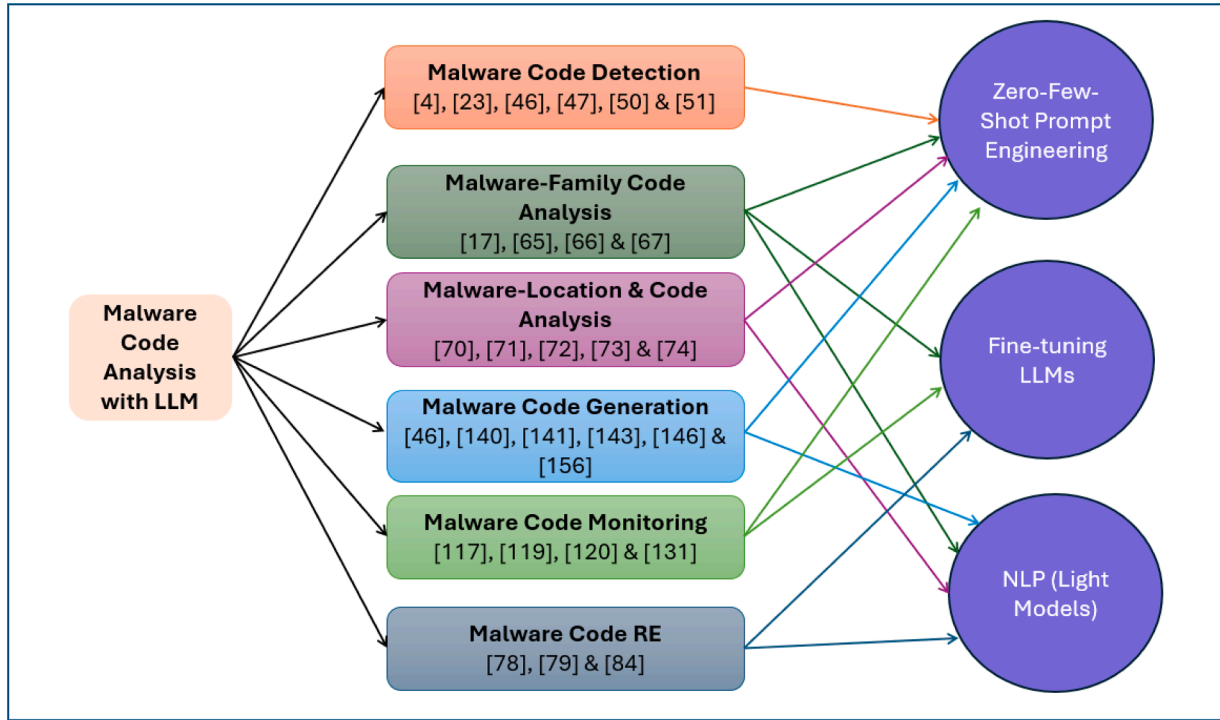


Fig. 4. A bibliometric analysis and visualization of potential LLM and NLP methods for malware code analysis.

Table 1

Some impressive LLM-based studies with reported performance metrics used for malware analysis. Acc = Accuracy, P = Precision, R = Recall, F1 = F1-score, AUC = Area Under Curve, IA = Instruction Accuracy, ESR = Execution Success Rate, ES = Effect sizes, KL = Kullback–Leibler Divergence, Van = Vanilla, API-Sc = API-Scoped, SecB = SecBench, TaintB = TaintBench, ED = Edit Distance.

Ref	Year	Application	Models	Dataset	Evaluation Metric Used
[47]	2024	Windows Malware Analysis	GPT-3.5-Turbo	Created with ChatGPT	EvadeRate = 37.6% ± 33.6%
[48]	2024	Windows Malware Analysis	GPT-4-Turbo	VirusTotal	TADA-Detect = 87.8%
[49]	2023	Windows Malware Analysis	GPT-3	200,000 PE files	EvasionGain = +25%
[50]	2024	Java Malware Analysis	WizardCoder	CVEfixes	F1 = 0.71 (bal.) / 0.27 (imb.); AUC = 0.69–0.86
[33]	2024	Java Malware Analysis	GPT-4	Java Zero-Trust Dataset	Metrics not reported
[51]	2024	Android Malware Analysis	ChatGPT / GPT	Kronodroid	Qualitative analysis only
[45]	2024	Android Malware Analysis	GPT-4o-mini	200 APKs	Acc = 77%; P = 0.76; R = 0.776; F1 = 0.768; Van = 49.5%; API-Sc = 56%
[52]	2024	Android Malware Analysis	ChatGPT, ERNIE, Qwen	-	IA = 0.93 / 0.83 / 0.92; ESR = 0.53 / 0.33 / 0.47
[5]	2024	Android Malware Analysis	GPT-4-1106-preview, Text-embedding-ada-002	AndroZoo	Acc = 97.15%; P = 97.03%; R = 97.39%; F1 = 97.21%; Ablation = 94.7–96.5%
[53]	2025	Android Malware Analysis	ChatGPT / GPT	-	KL = 0.30; ED = 3.22; ES = S:3 / M:6 / L:3
[54]	2025	Android Malware Analysis	GPT-3.5, GPT-4, Gemini-1.0, Claude-3	TaintBench Suite	P = 73.75–81.38%; R = 92.16–95.75%; F1 = 0.70–0.87; SecB = 0.81; TaintB = 0.51
[55]	2024	Website Malware Analysis	MobileBERT	25,796 websites	P = 0.95; R = 0.96; F1 = 0.95; Latency = 0.44s; Memory = 76MB
[56]	2024	Website Malware Analysis	BERT	-	Acc = 96.71–99.78%; P = 96.25–99.73%; R = 96.50–99.34%; ISCX Acc = 99.98%
[57]	2024	Website Analysis + Prompt Generation	BERT	1255 phishing prompts	Acc = 96% (web); 94% (Email); F1 = 0.95 (Claude); 0.91 (Bard)
[58]	2024	Website Malware Analysis	GPT-4, GPT-3.5, Gemini Pro, Llama-2-70B	1000 phishing/malware sites	P = 98.7%; R = 99.6%; AUC = 0.99

4.2.2. Which models are highlighted for fine-tuning LLMs for malware code detection?

Some works have been fine-tuned specifically for detecting malware using LLMs. The authors in [37], introduced Malt5, which was fine-tuned using the MalS dataset in combination with a benign pseudocode summarization dataset. Their underlying intuition was that the malicious semantic knowledge from malware source code summarization and the function patterns from benign pseudocode summarization, learned from these two datasets, could be transferred to generate malware pseudocode.

In [61], the authors fine-tuned the BERT embedding and introduced a deep learning-based approach for behavior-based malware analysis. APILI identified API calls linked to malware techniques in dynamic execution traces. It used multiple attention mechanisms to connect API calls, resources, and techniques, integrating adversary tactics, techniques, and procedures through a neural network.

In [43], the authors fine-tuned the BERT-base-uncased model to improve malware detection. They used Stacked BiLSTM and GPT-2 models to detect malicious code, creating language models from assembly instructions in benign and malicious PE files. Three datasets were formed: one for Document Level Analysis with Stacked BiLSTM, one for Sentence

Level Analysis with various models, and a third for custom pre-training using unlabelled instructions.

In [62], they evaluated the power of modern LLMs for a well-defined malware analysis task: deobfuscating malicious PowerShell scripts. They used OpenAI's GPT-4 (gpt-4-1106) and Google's Gemini "Pro" model, both accessed via their official APIs. Additionally, they considered two locally deployed LLMs: Meta's Code Llama Instruct (with 34B parameters), based on Llama 2 and fine-tuned on 500 billion tokens of source code data, specifically designed for processing large chunks of codes.

In [29] The authors fine-tuned the CodeGen-Mono-350M model [63], using Llama3-Instruct-8B as the teacher LLM. Llama3, being a highly capable code synthesizer, was ideal for bootstrapping the training dataset.

4.2.3. Pre-Training and fine-Tuning

Some prior works have examined the pre-training and fine-tuning steps for malware analysis based on LLM. In [64], the authors introduced m-BERT, a deep learning model based on the BERT architecture, to address the gap in Python source code vulnerability detection. They curated a dataset by combining malicious code from open sources with benign code from PyPI. In the pre-training phase, a standard BERT

Prompt

You have access to a simplified version of a malware website's source code. Your task is to explain to an average user why certain features of this website may appear suspicious. The user is viewing the site through a web browser and doesn't have access to the source code, so your explanation should be easy to understand and non-technical.

Guidance

Your explanation should help the user recognize and avoid potential security threats. Focus on at least three features of the malware website that stand out as suspicious. For each suspicious feature, follow this structure:

- Name the feature clearly (e.g., "Login Form").
- Explain the purpose of the feature and where it is located on the webpage (e.g., top, bottom, etc.).
- Highlight any suspicious text or wording that may appear.
- Point out any visual or functional aspects that might indicate the site's malicious intent.
- Keep in mind that this is a malware website, not an email.
- Be specific about the location of the feature within the webpage layout (e.g., "top of the page," "a certain button").

Output Structure

The explanation should be in a JSON format like this:

```
{
  "suspiciousFeatures": [
    {
      "purpose": "<feature purpose>",
      "location": "<feature location>",
      "aspects": "<suspected malicious aspects>"
    },
    {
      "purpose": "<another feature purpose>",
      "location": "<feature location>",
      "aspects": "<suspected malicious aspects>"
    }
  ]
}
```

Fig. 5. A sample prompt for malware code analysis using zero-shot approach for websites.

model was trained with a masked language model objective to acquire generalized representations of Python code, resulting in m-BERT.

4.2.4. Few-Shot and zero-Shot for malware code detection

Few-Shot and Zero-Shot learning are advanced techniques used in malware analysis with LLMs, enabling them to identify and analyze malware with minimal task-specific training data [34]. Few-Shot learning allows models to recognize malware patterns from just a few examples, helping them generalize and detect new variants of malware in an ever-changing cybersecurity landscape. Zero-Shot learning takes this further by enabling malware detection without direct examples, relying on contextual understanding and prior knowledge [10].

4.3. Comparative evaluation of general-purpose and domain-specific LLMs for malware analysis

Table 1 reveals clear performance stratification between general-purpose large language models (e.g., GPT-3.5, GPT-4, Gemini, Claude) and domain-specific models tailored for program or malware analysis (e.g., fine-tuned BERT, MobileBERT, AppPoet, WizardCoder). While general-purpose LLMs demonstrate strong reasoning and abstraction capabilities, the empirical results show that domain-specialized models consistently achieve higher and more stable classification performance on malware-centric tasks.

General-purpose LLMs. Models such as GPT-3.5, GPT-4, and Gemini appear in [45,47,54,58], where they demonstrate effective semantic understanding and cross-task adaptability. For example, GPT-4V in ChatPhishDetector [58] achieves near-perfect phishing detection with 98.7% precision and 99.6% recall, benefiting from its multimodal reasoning and large-scale pretraining on web data. However, in malware-specific tasks such as semantic categorization [45], general GPT-based models achieve moderate performance (77% accuracy), indicating that linguistic reasoning alone is insufficient for capturing low-level program behaviors.

Domain-specific pretrained models. In contrast, fine-tuned BERT-based systems [56,57] and the MobileBERT-based PhishLang [55] consistently achieve very high accuracy (>96%) with strong precision-recall balance. This performance gain arises from their task-aligned pretraining on structured security artifacts such as URLs, byte-level strings, and network indicators, which better reflect malware distribution statistics than natural language corpora. Similarly, WizardCoder in [50] exceeds generic code reasoning models due to its training on large-scale programming and vulnerability datasets.

Multi-view and multi-phase architectures. The highest performance is observed in AppPoet [5], which achieves 97.15% accuracy and a 97.21% F1-score. Unlike monolithic LLM prompting, AppPoet integrates multiple semantic views-including permissions, APIs, URL strings, and textual summaries-into a unified prompt engineering pipeline. This architectural fusion enables the model to jointly reason over static, behavioral, and semantic program cues, explaining its superior performance over single-view GPT-based categorizers [45].

Impact of fine-tuning and task specialization. The results further indicate that fine-tuning on domain-specific datasets provides substantial gains over purely zero-shot prompting. For example, WizardCoder [50] outperforms general-purpose counterparts in vulnerability detection due to exposure to labeled flaw patterns during training. Likewise, MobileBERT in PhishLang [55] achieves real-time inference with high precision, highlighting the importance of architectural efficiency for deployment-oriented cybersecurity systems.

Summary of performance drivers. Overall, the observed performance differences are driven by four primary factors: (i) domain-aligned pre-training data, (ii) task-specific fine-tuning, (iii) inclusion of program-structure features such as APIs and permissions, and (iv) multi-view semantic integration. General-purpose LLMs offer broad reasoning capacity and strong zero-shot generalization, whereas domain-specialized models consistently outperform them on tightly scoped malware classification and detection tasks. This explains why systems such as AppPoet, MobileBERT, and fine-tuned BERT variants dominate the upper range of Table 1.

5. LLM For malicious-Family code analysis

Classifying malware into families is essential for understanding shared behaviors, identifying code reuse, and enhancing threat intelligence. This section presents an overview of techniques used for malware family classification, focusing on both non-LLM and LLM-based approaches.

5.1. Non-LLM

In [65], the authors proposed an NLP-based approach consisting of three key components: Information Collection, Semantic Embedding, and Familial Classification. More specifically, they employed a hierarchical attention mechanism to extract higher-order semantic information, which helps generate representation vectors for malware samples.

In a similar work [66], authors conducted research on malware family classification using machine learning techniques to address the challenge of detecting polymorphic and variable malware variants, which traditional signature-based detection methods struggle to identify. They used a dataset of 65,536 malware samples from VirusShare, labeled based on the Kaspersky engine, and extracted features from three key aspects: bytecode features, assembler code features, and PE features. Through extensive feature selection and feature fusion processes, they refined the feature set and experimented with eight classifiers from Scikit-learn, including SVM, decision trees, and random forest.

In [67], the authors enhanced CDGDroid to classify and characterize Android malware families using deep learning. They extracted control-flow (CFGs) and data-flow graphs (DFGs), encoded them into matrices for classification, and used n-gram sequences for characterization.

5.2. LLM-Based

There are a few models for malware families based on transformers and LLM that we can mention, such as [18,68]. For example, In [18], the authors suggested a context-driven framework that used LLMs based on GPT-4omini/ChatGPT for Android malware detection, addressing challenges such as dataset biases, evolving threats, and the lack of explainability in traditional methods. They assisted manual audits by categorizing malware into six common families: Adware, Backdoor, PUA (Potentially Unwanted Applications), Riskware, Scareware, and Trojan, providing a structured approach to interpreting malware behaviors.

Also in [68] the authors propose a novel method for pre-training Transformer-based models when symbols are missing, using program analysis to extract contextual dependencies instead of relying on masked language modeling. Their model, a BERT-like Transformer with Regularized Multi-head Attention (RMA), incorporates both Masked Language Modeling (MLM) and Masked Dependence Modeling (MDM) for pre-training. This approach is applied to tasks like binary similarity, type inference, and malware family classification, using a dataset of 5484 samples from VirusTotal and VirusShare. The method aims to improve model performance in code analysis tasks without symbolic information.

6. LLM For malicious code location analysis

When malware is found in source code, pinpointing its location is crucial for effective removal and further analysis. Malware often hides

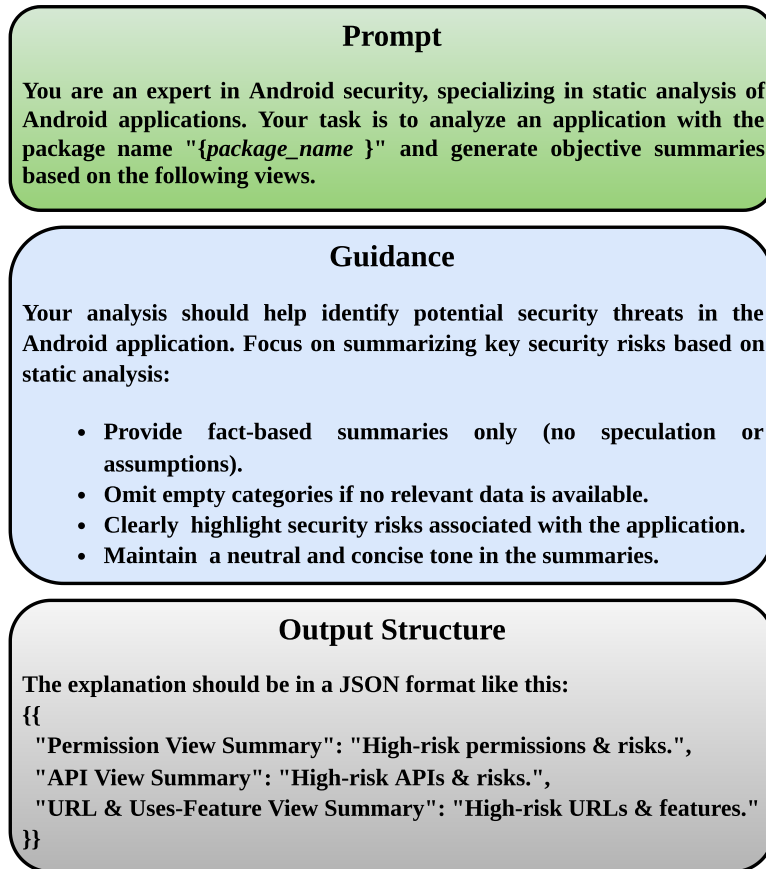


Fig. 6. A sample prompt of malware code analysis using zero-shot approach for Android apps.

within seemingly legitimate code, making it difficult to detect without careful examination. Malicious code typically resides in specific functions, modules, or files that exhibit unusual or suspicious behavior [69,70]. By analyzing the overall structure of the code, including function calls, imported libraries, and known malware patterns, we can identify these areas where the code performs potentially harmful actions.

6.1. Non-LLM: NLP and named entity recognition (NER) methods

NLP techniques, such as Named Entity Recognition (NER), can be applied to detect entities such as function names, variable names, library imports, or system commands [71]. Additionally, topic modeling can be used for malware analysis for opcode sequences, by treating a topic as a discrete distribution of opcodes [72,73].

6.2. LLM-Based: from transformers to large models

LLMs can be applied to identify the location of malware in source code by recognizing patterns and understanding the context of code. These models can detect suspicious behavior in code, such as abnormal API calls, network activity, or unusual file manipulations, which may indicate the presence of malware [74]. By analyzing the semantic structure of the code, LLMs can highlight areas where the code performs potentially harmful actions, such as using system-level commands or interacting with external servers, which are typical indicators of malicious code. For example, by training LLMs on large datasets of known malware samples, these models can identify code patterns that indicate malicious intent [75]. These patterns could include unusual control flow, suspicious API calls, or anomalous interactions with system resources.

7. LLM For malicious malware reverse engineering (RE)

LLMs have emerged as powerful tools for malware detection by analyzing PE and other executable files. These models leverage static analysis and reverse engineering techniques to extract meaningful patterns from binary structures, API calls, and assembly instructions. By learning from vast datasets of benign and malicious samples, LLMs can effectively classify threats, detect obfuscated code, and enhance cybersecurity defenses.

7.1. Non-LLM: Malware PE files and reverse engineering

Reverse engineering of malware-infected PE files is crucial for understanding their behavior, attack vectors, and obfuscation techniques. This process involves deconstructing the binary file to examine its structure, API calls, and embedded payloads [76]. Security researchers often use 1) disassemblers, 2) decompilers, and 3) debugging tools to analyze how malware interacts with system resources and executes malicious operations [77], such as IDA PRO [21] or Ghidra [22]. Additionally, they can help deobfuscate code by predicting transformations used by malware authors, making it easier to recover the original malicious intent hidden within PE files.

7.2. With LLM: Malware PE files and reverse engineering

LLMs enhance reverse engineering by assisting in the automatic annotation of disassembled code, suggesting potential functions of unknown code segments, and identifying common patterns found in malicious binaries[78]

From a reverse engineer's perspective, the next logical step is translating assembly language into a high-level language, which makes the

program significantly easier to read, understand, and modify. Unfortunately, this task is highly challenging for any tool, as compiling high-level source code into machine code results in a significant loss of information. For instance, it's impossible to determine the original high-level language of machine code just by examining it. While understanding specific characteristics of a compiler might assist a reverse engineer in identifying the machine code it produced, this approach is unreliable.

7.2.1. LLM-Based

The application of LLMs in low-level code generation, particularly for converting binary executables (such as PE files) into assembly, represents a significant advancement in reverse engineering, malware analysis, and security research [79]. Traditional disassembly and decompilation techniques rely on static analysis tools, which often require extensive manual efforts to interpret obfuscated or packed binaries. However, LLMs can augment these processes by providing semantic understanding of machine instructions, enabling more efficient reconstruction of assembly code from raw binary data [78]. By leveraging pattern recognition and deep learning capabilities, LLMs can enhance the accuracy of disassembled output and even assist in reconstructing high-level control flow from low-level instructions. This has profound implications for malware analysis, as it can expedite the identification of malicious logic, encoded payloads, and evasion techniques embedded within compiled binaries.

7.2.2. LLM-Based: Analysis of a packed malware binary

A packed malware binary in the context of a PE file refers to a type of executable file that has been intentionally compressed or encrypted to hide its true contents. The purpose of packing is to make it harder for security software, such as antivirus programs or manual reverse engineers, to identify malicious code [80]. Suppose a cybersecurity team is investigating a PE file that is suspected of being a malware sample. Traditional disassemblers, such as IDA Pro or Ghidra, are able to partially decode the file, but much of the code remains obfuscated or packed, making it difficult to analyze. The team decides to apply a LLM to aid in converting the packed binary into assembly code, uncovering the malware's behavior.

The analysis process begins by loading the packed binary into the investigation system. Due to the packing, which involves deliberately compressing or encrypting the file to obstruct analysis, manual unpacking would be time-intensive and require specialized expertise to identify the packing methods. To streamline this, a LLM is integrated with existing analysis tools. This model, fine-tuned on data involving packed executables and encryption techniques, can identify patterns associated with common malware packing methods, significantly accelerating the unpacking process.

Once integrated, the LLM utilizes its deep learning capabilities to decode the packed binary. It converts the raw binary into assembly code, which is far more readable and interpretable than the original machine-level instructions [62]. This transformation is key to understanding the program's behavior, as it provides a clearer view of the malware's operations. Additionally, the LLM employs semantic analysis to detect obfuscation techniques such as encryption, anti-debugging, and code manipulation, which are frequently used by malware to evade detection. The decoded assembly code produced by the LLM offers a detailed and more accessible representation of the malware's logic.

7.3. Decompilers & high-level language generation

Decompilers are designed to transform binary code into high-level source code, often targeting specific languages and compilers. However, they typically do not produce fully compilable source code. The pseudo-code generated is often harder to interpret than human-written code, although it is generally easier to understand than raw assembly. These tools aim to translate binary code into high-level source code, tailored to specific languages and compilers [81].

7.4. LLM-Assisted Deobfuscation of malware code

Hackers often obfuscate their scripts to avoid detection by security tools and analysts. This is commonly seen in malware, spyware, and other types of malicious code. When decompiling PE files, we may encounter obfuscated code as hackers use these techniques to hide their intentions. The goal is to make the code harder to analyze and reverse engineer, which complicates efforts to understand or neutralize the malicious payload. LLMs can help in automating the analysis of obfuscated code [62]. These models can assist security professionals by quickly identifying patterns and functionality within obfuscated scripts, even when they are heavily disguised. LLMs can also generate insights and suggestions on how to reverse obfuscation, making it easier to detect and mitigate malicious behavior [82]. They can speed up the process of code analysis and even automate some tasks that would be time-consuming for analysts, improving efficiency in cybersecurity.

In PE files and script-based malware, adversaries routinely apply multiple layers of obfuscation including identifier renaming, string encoding, control-flow distortion, API hiding, and packing to conceal malicious intent. Traditional deobfuscation requires significant manual effort, whereas recent studies show that LLMs can help unwind these transformations more systematically [62,82].

Identifier and variable-name obfuscation. Adversaries often replace meaningful identifiers with random tokens to hinder semantic understanding. LLMs can infer the intended semantics from surrounding code context, enabling reconstruction of human-readable variable and function names [62,82,83].

String and constant encoding. Malware commonly uses XOR, Base64, and custom encoders to conceal API names, URLs, and configuration data. LLMs can recognize encoder patterns, describe their effects, and assist in reconstructing decoded strings [84].

Control-flow obfuscation. Techniques such as opaque predicates, dead-code insertion, and flattened control graphs complicate reverse engineering. LLMs reason at a semantic level and can summarize the true execution path despite misleading control-flow constructs [85–87].

API hiding and dynamic resolution. Malware frequently resolves APIs dynamically using hashing, indirect calls, or custom PE parsing to avoid static inspection. LLMs can identify these patterns and map them to likely Windows API calls such as LoadLibraryA and GetProcAddress [88–90].

Packing and multi-layer obfuscation. While LLMs do not replace traditional unpackers, they assist by describing unpacking loops, decompression steps, and execution stubs, aiding analysts once partial extraction has occurred [84,91,92].

Illustrative example. Fig. 8 presents a comparison between two implementations that load and execute a malicious DLL. Code B contains clear, descriptive identifiers (e.g., loadLibraryFunc, libraryPath), whereas Code A employs obfuscated naming conventions such as _0x1, _0x2, and _0x3. This reflects a common tactic in malware: semantic clarity is intentionally sacrificed to reduce analyst comprehension. Despite identical functionality—calling LoadLibraryA and GetProcAddress—the obfuscation in Code A increases cognitive load for human reverse engineers. LLMs help bridge this gap by interpreting the semantic intent of such constructs, summarizing their purpose, and assisting in reconstructing the unobfuscated logic, thereby accelerating malware triage and analysis.

```

1 int main() {
2     system("gcc /tmp/malware_copy.c -o /tmp/
3         malware_copy");
4     system("/tmp/malware_copy &");
5     system("openssl enc -aes-256-cbc -salt -in /tmp/
6         important_file.txt
7         -out /tmp/encrypted_file.txt");
8     system("touch /tmp/.hidden_malware");
9     return 0;
}

```

A: Source Function

```

1 int fastcall main(int argc, const char argv, const
2     char envp) nn
3     system(gcc tmpmalwarecopy.c - o4 tmpmalwarecopy)
4     n system(tmpmalwarecopy)
5     system(openssl enc - aes - 256 - cbc - salt - in
6         tmpimportantfile.txt -
7         out2 tmpencryptedfile.txt) n system(touch tmp
8         .hiddenmalware) n

```

B: Decompiled function from a malware PE file

Fig. 7. Top-1 retrieved source function matched to a malware function query from a PE file.

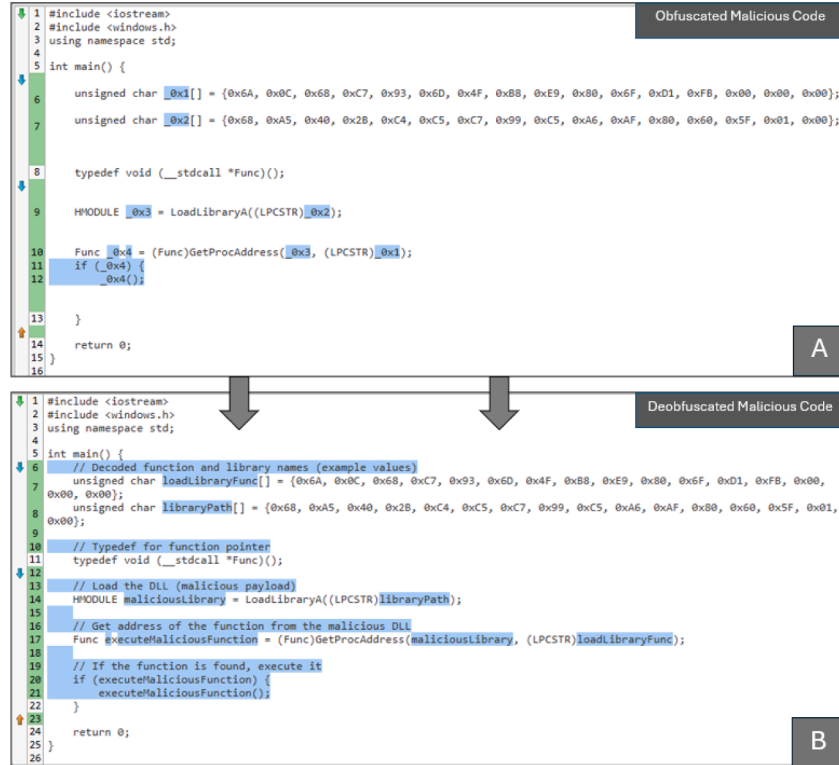


Fig. 8. Example of the deobfuscation techniques used in malicious code.

7.5. How well does the embedding model measure similarity between source functions and binaries?

Binary-to-source similarity relies on the ability of an embedding model to map heterogeneous representations—high-level source functions and compiled binary functions—into a shared semantic space. Fig. 7 illustrates an example where the model retrieves the correct Top-1 source function for a binary query, indicating strong semantic alignment.

Modern systems tokenize binary functions using multiple modalities: (i) *opcode sequences* capturing instruction-level semantics [93,94], (ii) *API call traces* and imported library symbols encoding higher-level functional behavior [88], (iii) *intermediate representations (IRs)* such as VEX or LLVM-IR that normalize compiler-dependent details [95], and (iv) *graph structures* such as control-flow graphs (CFGs) and program-dependence graphs (PDGs) [96,97]. These representations are converted into tokens using vocabularies that include instruction mnemonics, operand classes, symbolic constants, and API identifiers.

Source functions, by contrast, are tokenized using identifiers, operators, types, and syntactic constructs. Cross-modal alignment is achieved via contrastive learning, where each binary function is paired with its

original source implementation as a positive example and unrelated functions as negative examples. This CLIP-style dual-encoder optimization reduces cosine distance for true binary–source pairs while pushing apart unrelated functions [98,99]. As a result, the embedding space captures semantic similarity rather than textual or syntactic overlap, enabling robustness against compiler optimizations, inlining, register allocation, and minor obfuscation.

Recent work demonstrates that combining opcode-level signals with IR-level and API-aware tokens yields significant improvements in binary similarity search. SAFE [93], PalmTree [94], Trex [97], and Gemini [96] highlight that enriched semantic representations outperform raw mnemonic sequences. BinaryAI [98] integrates feature extraction, embedding-based retrieval, locality-aware refinement, and library detection, outperforming symbolic-diffing and syntactic-matching tools. These models achieve high precision in recovering function-level matches, demonstrating that embedding spaces grounded in instruction semantics and behavioral cues provide strong binary-to-source alignment.

Overall, embeddings measure binary-source similarity effectively when they capture both low-level execution semantics (opcodes, control-flow, IR) and higher-level functional behavior (API usage,

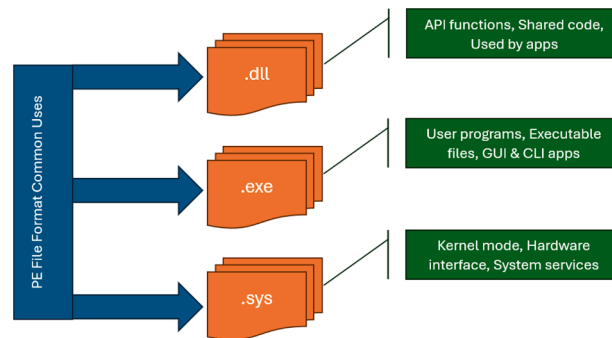


Fig. 9. Types of PE files/formats.

data-flow patterns). This synergy makes LLM-based or embedding-driven retrieval systems extremely powerful for reverse engineering, vulnerability triage, and software supply-chain assurance.

7.6. Transformative stages in reverse engineering and pre-processing

Regarding the Fig. 10, the malware code and reverse engineering can be divided into three key stages: 1) data collection and reprocessing, where raw executable files are gathered and transformed into meaningful features; 2) pre-processing and reverse engineering; 3) tokenization and embedding. These stages are discussed below:

7.6.1. Data collection and preprocessing

Regarding the first step in Fig. 10, we need to prepare and collect malware samples, specifically focusing on PE files. PE files are a standard binary format for executables, DLLs, and other system files on Windows operating systems. These files contain both machine code and metadata, making them a prime target for malware attacks. The prevalence of PE files in the Windows environment further enhances their attractiveness as a vector for malicious actors seeking to exploit vulnerabilities [100].

In particular, the structure of PE files often enables sophisticated techniques such as code injection, obfuscation, and packing, which can evade traditional detection methods. As shown in Fig. 9, there are various types of PE files, each serving different purposes, from executable files (.exe) to system driver files (.sys) and dynamic link libraries (.dll). This high susceptibility to exploitation highlights the need for robust analysis techniques to detect malicious code within PE files.

To ensure a diverse dataset that includes both benign and malicious samples, PE files are gathered from multiple sources. Malware repositories, such as VirusTotal [101], MalwareBazaar [102], ANY.RUN [103], and Hybrid Analysis, provide labeled samples from public and private databases. Real-world samples are collected from infected systems or honeypots [104] designed to attract malware, including suspicious Email attachments and drive-by downloads captured by web crawlers. Threat intelligence feeds [105,106] contribute executable samples shared by security researchers and organizations in response to recent malware campaigns. Additionally, legitimate software binaries are sourced from official platforms to serve as a baseline for comparison.

7.6.2. Malware reverse engineering and feature extraction

Regarding the second step in Fig. 10, we need to utilize decompiling/disassembling tools to generate low-level or high-level code. Disassembly, the process of translating machine-readable bytes into human-readable assembly instructions, is crucial for binary reverse engineering tasks such as binary rewriting and vulnerability detection [25]. A major challenge lies in correctly identifying instruction boundaries, as the mixed storage of code and data or variable-length instructions can make it difficult to determine where instructions begin. This often leads to errors in identifying instruction boundaries and the instructions that

follow. Tools like IDA Pro and Ghidra can heuristically convert binary code into C-like pseudo-code, but they often rely on heuristics and may struggle with complex or obfuscated binaries.

Opcode analysis and API call analysis are methods used to understand how software operates at different levels. Opcode analysis is low-level, focusing on the actual machine code and assembly instructions that the CPU executes, revealing details like memory manipulation and processor operations [107,108]. It's crucial for reverse engineering and malware analysis. In contrast, API call analysis is high-level, tracking how a program interacts with the operating system and libraries through system calls, such as file operations or network communication [109]. While opcode analysis examines how a program runs at the processor level, API call analysis reveals how it interacts with its environment. Together, they offer a comprehensive view of a program's behavior.

7.6.3. Embedding and tokenization

Regarding the third step in Fig. 10, to feed transformers or LLM models, it is essential to tokenize and embed the samples. Based on previous research [110], there are different types of samples used for embedding, including opcode sequences [111,112], API calls [113,114], byte-level representations [115,116], and instruction-level details [117,118]. Each of these types provides a unique perspective on the data, and embedding them effectively helps the model understand and process the information efficiently for various tasks.

7.7. Importance of LLMs for malware PE file analysis

LLMs provide a groundbreaking solution to these challenges in malware code detection through static analysis of PE files. Unlike traditional models, LLMs possess contextual understanding, enabling them to analyze and interpret raw binary and code structures without extensive feature engineering. Their ability to generalize beyond predefined feature sets makes them highly effective against zero-day threats and sophisticated obfuscation techniques. Additionally, LLMs enhance explainability and adaptability, addressing the transparency concerns often associated with traditional machine learning models [119]. Table 2 presents a comparative analysis of various non-LLM-based approaches in malware detection, specifically focusing on the static analysis of PE files. The studies listed highlight a range of traditional machine learning and deep learning methodologies, each with distinct advantages and limitations. While these approaches demonstrate high accuracy, scalability, and efficiency, they also face critical challenges that limit their effectiveness in handling complex and evolving malware threats. Regarding the Table, key observations from the table reveal that conventional methods such as transfer learning, ensemble learning, and feature-based classifiers exhibit strong detection capabilities. However, they also suffer from major drawbacks, including feature representation limitations, and computational overhead.

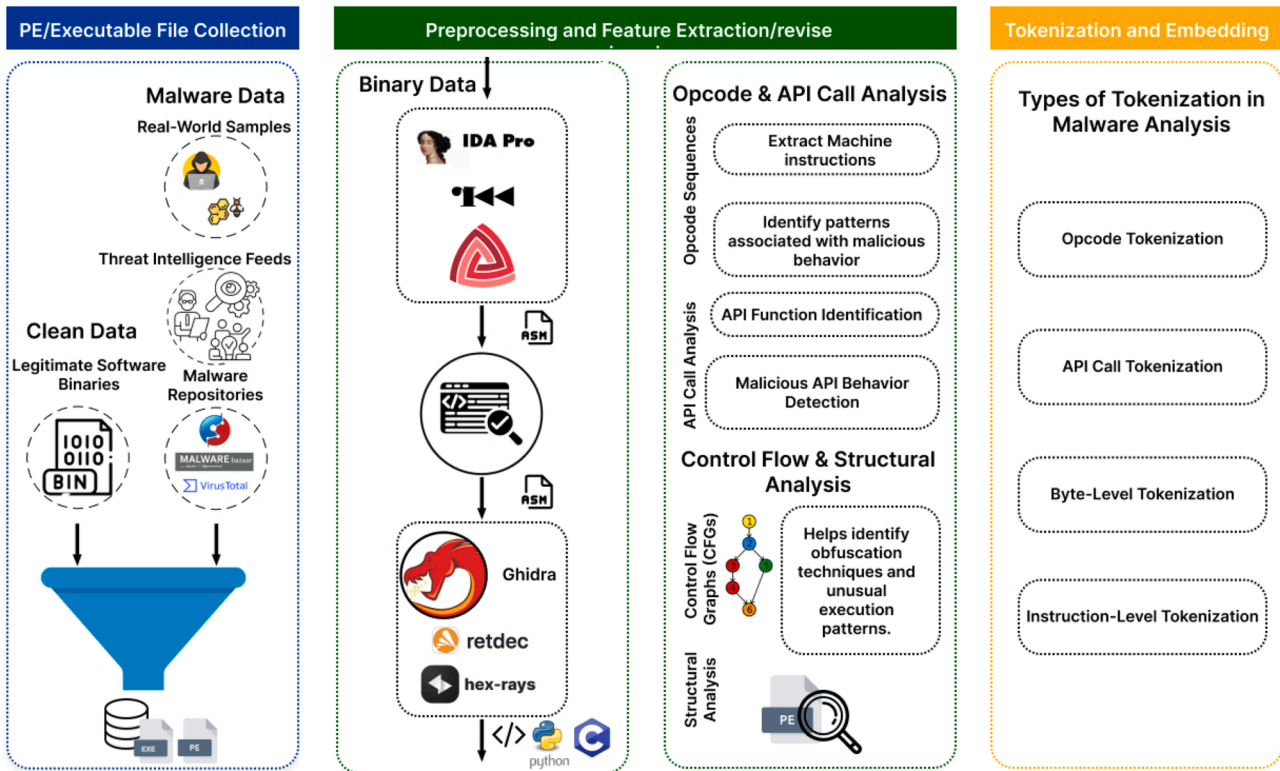


Fig. 10. A general overview of pre-processing and reverse engineering for disseminating data on PE files.

8. LLM For code monitoring/inspection and malware prevention

While code reviews/inspections aim to prevent vulnerabilities from being introduced into software during development, malware analysis is focused on understanding and mitigating threats that exploit vulnerabilities. Malware code analysis is a reactive process that involves examining malicious code after it has been identified. The goal is to understand how the malware works, how it exploits vulnerabilities, and how to defend against it.

8.1. Identification of potential defects

The integration of LLMs into software development has opened new avenues for enhancing the efficiency and accuracy of critical tasks, including code review and inspection. Traditionally, these processes rely on manual efforts and static analysis tools, which, while effective to some extent, often face challenges such as high false positive rates, poor generalization, and coarse detection granularity. LLMs, with their advanced NLP capabilities, present a transformative opportunity to address these limitations. They offer not only the ability to identify potential defects in code but also the capacity to provide detailed, human-readable explanations and actionable suggestions for remediation. This dual capability positions LLMs as a promising tool for streamlining code review and inspection processes while improving their reliability and comprehensiveness [120,121].

Recent studies have demonstrated that LLMs outperform traditional static analysis tools in identifying security defects and other vulnerabilities during code review. For instance, GPT-4 has shown remarkable potential in detecting defects, especially in smaller codebases or those written by less experienced developers. However, these studies also highlight significant challenges, including the generation of verbose responses and inconsistency in outputs. Prompt engineering techniques, such as few-shot and chain-of-thought prompting, have been explored

to enhance LLM performance, but the variability in response quality remains a barrier to their widespread adoption [121,122].

8.2. Malware prevention with vulnerability analysis

Beyond vulnerability detection, LLMs have found utility in several related tasks that can fall under the category of code review in the software development lifecycle, such as defect detection, test generation, and bug analysis. These applications illustrate the versatility of LLMs but also underline the need for targeted evaluations to understand their specific strengths and limitations in code review. This section aims to provide a concise overview of how LLMs are being applied to code review and inspection, focusing on their capabilities, challenges, and practical implications for software development.

Among the above-mentioned tasks, vulnerability detection is the most crucial and has gained a lot of attention as a use case for LLMs. Vulnerability detection is a vital process in software development, focusing on identifying weaknesses in code that could be exploited by attackers. This task is closely tied to standards like Common Vulnerabilities and Exposures (CVE) and Common Weakness Enumeration (CWE), which provide a framework for categorizing and documenting security flaws. While traditional tools often struggle with precision and context-awareness, LLMs are emerging as a powerful alternative. By analyzing code with these established frameworks, LLMs can detect vulnerabilities more effectively and offer detailed explanations to help developers address security risks earlier and with greater accuracy.

One of the seminal works on using LLMs in vulnerability detection is done by Thapa et. al. [123]. Their work explores the application of transformer-based models like BERT, GPT-2, and CodeBERT for identifying vulnerabilities in C and C++ source code. It introduces a systematic framework that integrates pre-training, fine-tuning, and inference to enable effective vulnerability detection. The framework leverages code gadgets-semantically related slices of code extracted based on data dependencies-to serve as inputs for the transformer models,

Table 2
Highlighting the value of LLMs in contrast to Non-LLM models.

Study	Non-LLM Approach	Cons	Why LLM might be a better choice?
[239]	CNN + Inception architecture	Feature Representation Limits Computational Cost Vulnerability to Adversarial Attacks	Contextual Awareness Zero-Day Detection Explainability
[240]	TL + EML	Lack of Context Awareness Computational Complexity Feature Representation Limits	Contextual Understanding Adaptability Explainability
[241]	PE Header Analysis + YARA Rules Integration	Limited Behavioral Insight Dependence on Known Patterns	Contextual Understanding Adaptability to Novel Threats Explainability
[242]	DNA Encoding of Features + Bioinformatics Analysis	Computational Complexity Scalability Concerns Interpretability	Direct Analysis of Code and Behavior Scalability and Efficiency Adaptability Explainability
[243]	Static Analysis of PE Headers + Feature Extraction	Potential for Evasion Limited Behavioral Insight	Contextual Analysis Adaptability Comprehensive Detection
[244]	Feature Extraction + RF/DT/ Gradient Boosting and AdaBoost/ Bayes Theorem + Benchmark Evaluation	Feature Selection Complexity Model Generalization Computational Overhead	Advanced Pattern Recognition Adaptability to New Threats Reduced Feature Engineering
[245]	Dataset Development + Feature Selection via Swarm Optimization + KNN/NC/RF/ GNB/SVM/DT	Static Analysis Limitations Optimization Overhead	Contextual Understanding Adaptability Reduced Need for Feature Engineering
[246]	Feature Extraction + LR/SVM/KNN/Trees/ feature extraction	Evasion Techniques Limited Behavioral Insight	Contextual Understanding Adaptability Reduced Feature Engineering
[247]	+ CNN	Limited Behavioral Insight Evasion Techniques	Contextual Understanding Adaptability Reduced Feature Engineering
[248]	Feature Extraction + LSTM Network	Evasion Techniques Limited Behavioral Insight	Comprehensive Data Understanding Automated Feature Extraction Scalability

enabling a structured approach to analyzing code vulnerabilities. The system flow involves pre-training on large text corpora, fine-tuning with labeled vulnerability datasets using added classification layers, and inference for predicting vulnerabilities in new code samples. This approach enables the models to adapt effectively to software-specific tasks while leveraging their pre-trained knowledge.

In one such related work by Szabo et al. [124], authors explored using GPT models, particularly GPT-3.5 and GPT-4, to detect vulnerabilities in Angular-based web applications. It focuses on CWE-653 vulnerabilities related to the improper isolation of sensitive data handling. The study introduced a sensitivity classification framework for data and tested the pipeline on large open-source Angular projects.

Another work by Noever et al. [125], investigated the GPT-4's ability to detect and fix software vulnerabilities, comparing its performance with traditional tools like Snyk and Fortify. GPT-4 identified four times more vulnerabilities than Snyk across 129 code samples in eight programming languages and effectively reduced vulnerabilities by 90% after applying its fixes. While demonstrating strong detection and self-correction capabilities, the study highlights limitations in handling large system-level codebases and occasional false negatives. The findings suggest GPT-4 can complement traditional static analyzers, offering a promising but still developing approach to software vulnerability management.

Another notable study that evaluated the performance of LLMs in vulnerability detection is presented in [126]. The authors introduced *VulBench*, a comprehensive benchmark dataset that aggregates real-

world and Capture-the-Flag (CTF) vulnerabilities. The study conducts a large-scale evaluation of 16 LLMs, including GPT-3.5, GPT-4, and several open-access models, comparing their effectiveness against deep learning-based methods and traditional static analysis tools. The evaluation considers both binary classification (whether a function is vulnerable) and multi-class classification (identifying specific vulnerability types). Results show that while LLMs outperform traditional deep learning models and static analyzers in controlled settings, their effectiveness declines in real-world scenarios requiring broader code context. The study also highlights the impact of few-shot prompting, showing that additional context improves detection rates. However, dataset quality remains a major challenge, as existing vulnerability datasets often contain inconsistencies. The findings suggest that while LLMs hold promise for vulnerability detection, they are best utilized in combination with static analysis and fuzzing techniques to enhance accuracy and reliability.

In [50], the authors focused on fine-tuning LLMs for detecting vulnerabilities in Java source code. First, they start by selecting a suitable from open-source LLM models trained on large code corpora. They have selected WizardCoder [127] which is compatible with transformer training frameworks and works better in answering questions with Yes or NO. Also, to address the challenge of performing classification tasks with LLM models they proposed a modified cross-entropy loss. The dataset they used in their work, was constructed from three open-source datasets: CVEfixes [128], a Manually-Curated Dataset [129], and VCMATCH [130]. Their comprehensive evaluation study showed the

Table 3
LLM-Based approaches for vulnerability detection.

Ref	LLM Models	Language	Datasets
[123]	GPT-2, BERT, CodeBERT, DistilBERT, RoBERTa	C, C++	VulDeePecker, SeVC
[124]	GPT-3.5, GPT-4	Angular	GitHub
[125]	GPT-4	C, Javascript, Java, Go, C#, PHP, Ruby	Codebase of Security Vulnerabilities
[126]	GPT-3.5, GPT-4, Llama-2, CodeLlama, Vicuna	C	VulBench
[50]	WizardCoder	Java	CVEfixes, VCMATCH
[131]	CodeBERT, code-davinci, text-davinci	C++, Java, Javascript, C#, Go, Ruby, Python	GitHub

fine-tuned WizardCoder model higher performance compared to previous CodeBERT-based models used for the vulnerability detection task.

Chan et. al. [131], explored the use of transformer-based models for detecting vulnerabilities in code as developers write it, a process known as EditTime detection. Unlike traditional methods that analyze fully compiled code, this approach allows for real-time vulnerability detection, reducing the time between introducing and mitigating security risks. The authors trained models on a large dataset of vulnerable code snippets and experimented with zero-shot, few-shot, and fine-tuning techniques applied to CodeBERT, code-davinci-002, and text-davinci-003. Their study shows that fine-tuning provides a strong balance between accuracy and reliability, while zero-shot and few-shot learning offer more flexible but less precise detection. The research also highlights the risks of AI-generated code, showing that automated completions often introduce vulnerabilities. They also deployed their framework as a VSCode extension, significantly reducing vulnerabilities in developer-edited code and demonstrating the potential to integrate LLM-based security tools directly into the development workflow. Table 3, present a summary of works that incorporated LLMs in vulnerability detection tasks.

8.3. API Call inspection

Application Programming Interfaces (APIs) are essential to modern software development, enabling seamless interaction between diverse systems and services. As APIs grow in complexity, ensuring their correctness, reliability, and security has become increasingly challenging. Traditional API testing requires significant manual efforts to create and maintain comprehensive test suites, especially when dealing with complex data dependencies and evolving specifications. To address these challenges, recent research has explored the use of generative AI, particularly language models, to automate test generation, reduce human effort, and improve test coverage.

REST APIs, in particular, have become central to web services, microservice architectures, and the broader API economy. Automated REST API testing has emerged as a critical area of research, focusing on generating test cases from specifications like OpenAPI using strategies such as random input generation, evolutionary algorithms, and dependency analysis. However, existing tools often struggle with generating valid inputs, handling inter-request dependencies, and coping with incomplete or inconsistent specifications-leading to limited code coverage and missed faults. Recent work has explored solutions such as leveraging NLP to extract useful information from documentation, improving dependency inference, and incorporating dynamic feedback to create more robust and effective testing approaches [132].

In [133], the authors present RESTGPT, a novel approach that enhances REST API testing by extracting machine-readable rules from human-readable descriptions in OpenAPI specifications. Their framework, powered by GPT-3.5 Turbo, introduces a Rule Generator module that identifies four types of constraints: operational constraints, parameter constraints, parameter type and format, and parameter examples. Using carefully crafted prompts, RESTGPT improves API specifications by generating relevant parameter values and constraints, significantly outperforming existing techniques in both rule extraction accuracy and input value generation.

APITestGenie [134] is an LLM-based framework designed to automatically generate executable REST API test scripts from natural language business requirements and OpenAPI specifications. By leveraging the contextual understanding capabilities of LLMs, the tool creates end-to-end integration tests simulating real-world scenarios, even across multiple API endpoints. APITestGenie achieved a test validity rate of up to 80% through iterative prompt refinement and employed Retrieval-Augmented Generation (RAG) to handle large API specifications. While it enhances tester productivity, the tool is positioned as a supportive assistant requiring human validation before deployment in production environments.

In another work, LlamaRestTest [135] introduces a resource-efficient approach to REST API testing by fine-tuning small language models, specifically the Llama3-8B model, to identify parameter dependencies and generate realistic test inputs. Unlike static specification-based methods, LlamaRestTest actively incorporates feedback from server responses during test execution to iteratively refine requests, ensuring higher accuracy and compliance with complex parameter constraints. Through fine-tuning and quantization, LlamaRestTest outperformed larger models and existing testing tools in both code coverage and fault detection, offering a scalable and cost-effective solution for dynamic and comprehensive API testing.

8.4. Code smells and pattern analysis

Code smells indicate poor coding practices that increase software complexity and maintenance challenges, often leading to security vulnerabilities. While not immediate defects, issues like long methods, excessive coupling, and global variables can obscure logic and make exploitation easier. Studies show that smells such as long parameter lists and complex class structures correlate with security flaws [136]. In languages like JavaScript, security-related smells such as hardcoded credentials, excessive global variables, and improper error handling can lead to injection attacks, unauthorized access, and data breaches [137]. Detecting and refactoring these smells early helps mitigate security risks and improve software robustness.

To address the limitations of traditional code smell detection, researchers have integrated machine learning (ML) and deep learning (DL) to enhance detection accuracy and reduce the subjectivity of heuristic-based methods. Unlike manual threshold-setting approaches, ML and DL models learn to differentiate between smelly and non-smelly code, mitigating performance instability. The use of LLMs in code smell detection has gained increasing attention in recent years, leveraging their advanced NLP capabilities and ability to understand complex software patterns. Traditional code smell detection methods relied on heuristic-based approaches, machine learning (ML), and deep learning (DL) models, which often struggled with subjectivity, high computational requirements, and limited generalization [138]. Recent studies have explored prompt-based LLMs, fine-tuned LLMs, and hybrid approaches that integrate expert toolsets to improve detection accuracy and efficiency.

A key development in this space is the exploration of Parameter-Efficient Fine-Tuning (PEFT) methods to adapt LLMs for method-level code smell detection [138]. Unlike full fine-tuning, PEFT techniques such as LoRA, prefix tuning, and prompt tuning have demonstrated comparable or superior performance while consuming fewer computational

resources. However, findings suggest that small LMs (under 1B parameters) can sometimes outperform larger LLMs for specific method-level smell detection tasks. This suggests that task-specific adaptations, rather than model size alone, play a crucial role in optimizing LLM-based detectors.

Another promising approach integrates LLMs with expert toolsets for code smell detection and refactoring. The iSMELL framework employs a Mixture of Experts (MoE) architecture, selecting the most suitable expert tool for each smell type and using LLMs to refine refactoring suggestions [139]. This hybrid approach addresses LLMs' limitations, such as difficulty in repository-level knowledge retrieval, token constraints, and dynamic source code analysis. Experimental results show that iSMELL outperforms baseline LLMs and traditional heuristic-based tools in detecting complex smells like Refused Bequest, God Class, and Feature Envy.

In [140] authors evaluated LLMs for detecting code smells in the Machine Learning Code Quality (MLCQ) dataset [141]. Their study comparing GPT-4 and Llama found that prompt tuning significantly enhances LLM performance in detecting and classifying code smells. However, challenges remain in ensuring consistent detection across different smell types, as some LLMs struggle with distinguishing nuanced smells, leading to false positives and inconsistencies.

In the context of AI-powered code reviews, LLMs have been integrated into automated review systems to detect code smells and recommend improvements in real-time [142]. These systems leverage LLMs' ability to predict future code issues, identify design flaws, and enhance software maintainability. Similarly, research on LLMs for test smell detection has shown promising results, with models like ChatGPT-4 identifying 70% of test smell types across multiple programming languages [143].

9. LLM-Generated Malicious code

Recently, LLMs have been utilized to generate code for users. It facilitates generating code effortlessly and saves time. However, it has increased the risk of cyber threats to the community. The rising use of LLMs for code generation brings into question their ability to produce reliable and trustworthy code [144]. Attackers can use LLMs to generate malicious code automatically, endangering many software service providers (SSPs). LLMs are akin to a double-edged sword: in the hands of a bodyguard, they can offer protection, but if wielded by an adversary, they can cause harm [145]. This situation calls for an investigation to understand the dynamics of malicious code generation by LLMs and detect the threats effectively. Notably, the existing academic research on malware generation using LLMs remains scarce, with only a limited number of studies addressing this issue [146]. It is intriguing to explore how existing techniques address the challenges posed by LLMs in restricting malware generation. Several researchers have observed that LLMs do not generate malicious code solely based on the direct prompt given that asks to create one [146,147]. LLMs pose several limitations to restrict the generation of malicious content as discussed in the subsection below:

9.1. Obstacles in generating malware using LLMs

Regardless of the ability of LLMs to generate code, there are several hurdles and challenges in using them to synthesize fully functional malware. These obstacles arise due to ethical safeguards, AI limitations, and technical constraints. Some of the obstacles are listed below:

- LLMs incorporate built-in safety mechanisms to prevent the generation of malicious code [147]. If a prompt is identified as harmful, the model either refuses to respond or generates a generic or incomplete answer.
- Jailbreak prompts as discussed in subsection 9.2.3 can occasionally circumvent safeguards; however, models are continuously updated

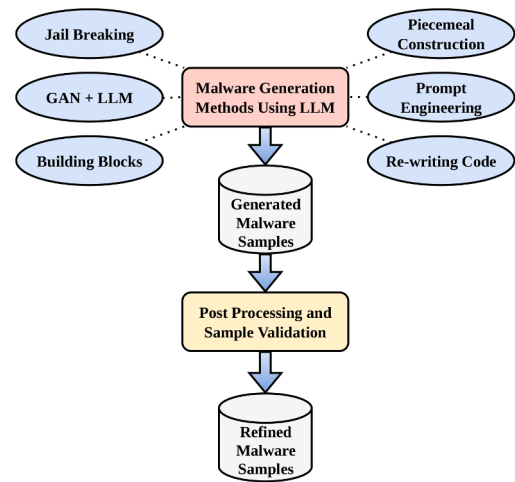


Fig. 11. A general view of LLM-based malware generation techniques to generate malware sample datasets.

to prevent such exploits [148]. OpenAI and other developers actively track and refine restrictions to block prompts requesting malware, hacking tools, or unauthorized scripts.

- LLMs generate code based on patterns learned from data but lack a deep understanding of complex exploit development, privilege escalation, or advanced evasion techniques. This absence of contextual awareness limits their ability to create sophisticated malware autonomously [149].
- LLMs can explain program execution flow for simple programs, their performance declines with increasing code complexity, particularly when dealing with non-trivial logical and arithmetic operations, non-primitive types, and API calls [150].
- Developing and deploying malware is illegal under various cybercrime laws, and using AI for such purposes raises significant ethical concerns [151]. The potential for AI-generated content to be used maliciously has been a topic of discussion among experts and legislators, emphasizing the need for responsible AI development and regulation [152].

9.2. Techniques applied for malware generation

LLMs are highly speculated to cause a cybersecurity threat [147]. Fig. 11 shows the latest techniques used in literature to generate malicious code. Additionally, Table 4 provides an overview of research publications that explore the use of LLMs for malware code generation. It categorizes studies based on the techniques used, the LLM models involved, the datasets referenced, the intended applications of generated malware, and the effectiveness of security detection mechanisms against such threats. Let's discuss the malware generation techniques in the Subsections below:

9.2.1. Prompt engineering to bypass AI constraints

As mentioned above, LLMs impose safety and ethical restrictions on generating malicious code. Charan et al. [153] proposed prompt engineering techniques to bypass these safeguards and generate malware capable of concealing PowerShell scripts while executing them on a pre-defined schedule. This approach suggests employing LLMs to generate payloads for executing cyber attacks. The study demonstrates the high efficiency of LLMs by producing executable code for the top 10 MITRE weaknesses identified in 2022 using ChatGPT [154] and Bard [155]. Furthermore, LLM-generated payloads are often more sophisticated and precisely targeted compared to those crafted manually.

9.2.2. Malware generation using building blocks

The building blocks approach to malware generation explored in the article [146] presents a modular methodology where LLMs like GPT-3 generate malware in small functional segments rather than as a single, complete piece of code. This approach is highly effective because while GPT-3 struggles to create entire malware from a single prompt, it excels at producing individual malicious functions when requested separately. The attackers can break down complex malware functionality into discrete building blocks such as file manipulation, process injection, privilege escalation, and persistence mechanisms. These functional components can then be manually or programmatically integrated into a fully operational malware sample.

Furthermore, GPT-3's ability to generate multiple implementations of the same behavior using different APIs helps evade signature-based detection systems. The paper demonstrates this approach by generating thousands of malware variants, some of which were detected by as few as 4 out of 55 antivirus engines on VirusTotal. This highlights the dual-use nature of LLMs, where their text-generation capabilities can be exploited to automate and scale malware development, reducing the skill barrier for cybercriminals.

9.2.3. Malware generation using jailbreaking

Jailbreaking is one of the recent strategies that allows the DAN (Do Anything Now) prompts to bar the restrictions of generative models such as ChatGPT [156,157]. More specifically, prompt engineering is employed to jailbreak ChatGPT [158]. Many researchers have explored jailbreaking attacks to exploit LLMs, such as [159,160].

Pa et al. [147] utilized jailbreaking prompts to bypass moderation controls and successfully generate functional malware and attack tools. The authors confirm that ChatGPT [154], text-davinci-003 [161], and Auto-GPT [162] can be exploited using jailbreaking prompts to bypass safety restrictions, iterative prompt refinement to improve malicious code generation, and automated multi-step prompt generation using Auto-GPT. The researchers successfully developed seven types of malware and two attack tools, including ransomware, worms, keyloggers, fileless malware, and AI-powered phishing mailers. The study highlights that jailbreaking ChatGPT with specific prompts significantly increases the likelihood of generating malicious code, while Auto-GPT autonomously refines its prompts, making it a powerful tool for malware development. The study found that malware generated using GPT-3.5-turbo exhibited low detection rates across various antivirus (AV) and endpoint detection and response (EDR) solutions. Additionally, Linux-based malware, including IoT worms and Telnet brute-force tools, had detection rates below 3%, making them particularly effective at bypassing traditional security measures. These results highlight the significant evasion capabilities of LLM-generated malware and the urgent need for advanced security strategies to mitigate AI-driven attacks. Despite this, the model's outputs contained frequent errors, requiring manual correction to execute attacks successfully.

9.2.4. Piecemeal construction

The concept of piecemeal construction in malware generation, as demonstrated by Monje et al. [163] in their study, highlighted a method of bypassing AI guardrails by requesting individual benign code snippets that, when combined, form a fully functional malware program. Instead of directly asking ChatGPT to generate a ransomware script - an action that would trigger OpenAI's ethical safeguards - the authors successfully circumvented restrictions by breaking down their requests into multiple, seemingly harmless programming tasks. These included generating encryption and decryption functions, creating a pop-up window for ransom demands, and monitoring cryptocurrency payments. The researchers constructed a functional ransomware program by stitching together these individual scripts, demonstrating a significant security loophole in LLM-based AI systems. This piecemeal approach not only exposes the vulnerabilities of LLMs in content moderation but also

raises concerns about the misuse of generative AI for malicious cyber activities.

9.2.5. Re-writing code for generating malware variants

Regarding code generation using various techniques, some researchers have explored the potential of LLMs in generating LLM-assisted malware variants. These variants can be created by rewriting the original source code of malware samples using LLMs.

In this direction, Huang et al. [47] investigated the effectiveness of LLMs, specifically OpenAI's GPT-3.5-Turbo, in generating malware variants and their ability to evade detection by antivirus software. The researchers developed 400 malware variants from four different types of malware (worm, keylogger, ransomware, and fileless malware) by modifying their source code using GPT-3.5-Turbo.

For each type of malware, 100 variants were generated, 211 of them compiled into fully functional executables. The similarity between the original and generated malware was assessed using BLEU scores and binary similarity metrics. The study also examines antivirus detection methods, evaluating six antivirus solutions using 160 malware variants. Detection techniques included both file-based scanning and behavior monitoring. The results indicate that fileless malware had the lowest detection rate, while ransomware was more likely to be identified through behavioral analysis. The worms and keyloggers exhibited varying evasion rates for each antivirus software, with some solutions not detecting certain variants.

Similarly, Botacin et al. [146] suggested that GPT-3 can produce different versions of the same malware by rewriting existing malicious code in multiple ways. The study generated 4820 functional malware variants, demonstrating GPT-3's ability to produce diverse and evasive malware samples. The detection rates of these variants varied significantly, and some samples evaded detection by most antivirus engines.

9.2.6. GAN And LLM for malware generation

Researchers have adapted LLMs and other generative AI models for various cybersecurity applications. For example, Devadiga et al. [49] proposed a GLEAM model (GAN and LLM for Evasive Adversarial Malware) utilizing GPT-3 for the generation of synthetic malware, particularly in the creation of hexadecimal representations of malware samples. LLM-generated malware was tested against machine learning-based malware detectors, and the results showed that GPT-3-generated malware had a higher evasion rate than GAN-generated samples. The study demonstrated an increase in the evasion rate when GPT-3-generated samples were used, highlighting its potential threat in adversarial malware generation.

10. Top LLM-Based models for malware analysis

In this section, we explore LLMs that demonstrate exceptional capabilities in malware code analysis. These models can be categorized into two main groups. The first group includes general-purpose LLMs, which are designed for a wide range of tasks and exhibit strong natural language and code processing abilities. While not specifically trained for malware detection, these models can be adapted for malware analysis by leveraging their foundational knowledge of programming languages and code structures. The second group consists of specialized offensive AI models that are specifically developed or fine-tuned for malicious purposes such as generating phishing attacks, obfuscating code, or automating cyberattacks, enabling them to detect, analyze, and classify malicious code with greater accuracy. These specialized models are particularly valuable for cybersecurity research, as they enhance the ability to identify evasive threats and emerging attack patterns. Fig. 12 illustrates the malicious exploitation of LLMs, categorizing them into general-purpose LLMs and specialized malware LLMs, along with their applications in cybersecurity threats. It maps how various LLMs contribute to different malware-related tasks, including phishing, malware detection evasion, code obfuscation, and automated cyberattacks.

Table 4

Publications corresponding to LLMs used for malware code generation.

Paper/Year	Techniques used		Mitre Tactics		LLM model(s)		Dataset Used		Application		Security Detection	
Pa et al. 2023 [147]		Jailbreaking		Initial access-Execution-Defense Evasion		ChatGPT, Davinci-003, GPT		Text-Auto	N/A	Generate malware and attack tools	Partially detected (low detection rates)	
Botacin et al. 2023 [146]		Building Blocks		Execution		GPT-3		Malware samples [164]	samples	Code generation and DLL injection	Some codes are detected by antiviruses; some are not	
Charan et al. 2023 [153]		Prompt Engineering		Top 10 Mitre Techniques from Red Report 2023 [165]		ChatGPT, Bard		Google	The Red Report 2023 [165]	Malicious Code Generation	Detection evasion in controlled sandbox environment	
Monje et al. 2023 [163]		Piecemeal Construction		Execution		GPT-3		System files		Ransomware generation	Some codes are detected by antiviruses; some are not	
Liu et al. 2023 [166]		Jailbreaking		Execution		GPT-3		Custom-built scenario & jailbreak prompt datasets		Identification of effective jailbreak prompts for malware generation	Some codes are detected by antiviruses; some are not	
Huang et al. 2024 [47]		Prompting LLM to re-write code		Initial access-Execution-Defense Evasion-Credential Access		GPT-3.5-Turbo		Malware samples (source unspecified)		Malware Variants Generation	Partially effective at evading antivirus detection	
Devadiga et al. 2023 [49]		GAN based attacks		Execution-Defense Evasion-Credential Access		GPT-3		PE Malware Machine Learning Dataset [167]		Evasive Malware Samples Generation	Partially effective at evading antivirus detection	

10.1. Maliciously exploited general-Purpose LLMs

Maliciously Exploited LLMs refer to the misuse of advanced AI-driven text generation systems for cybercrime, misinformation, and automated attacks. While GPT-4 [168], DeepSeek [17], Claude [169], and Gemini are designed for productive and ethical applications, they can be manipulated to generate harmful content, including phishing emails, malware code, misinformation campaigns, and social engineering scripts. Cybercriminals exploit vulnerabilities in LLMs by bypassing safety filters using adversarial prompts, jailbreak techniques, and fine-tuning attacks, enabling models to produce highly deceptive and malicious responses. A growing concern is the use of LLMs in malware development, where models assist in code obfuscation, polymorphic malware generation, and automated exploit scripting, significantly lowering the skill barrier for cybercriminals. Additionally, LLM-driven social engineering enables attackers to craft personalized phishing messages and fraudulent interactions, making them more convincing and harder to detect. The exploitation of LLMs poses serious security risks, necessitating stronger AI guardrails, real-time monitoring, and regulatory frameworks to mitigate their misuse in cybersecurity threats.

10.1.1. Openai models

The advent of OpenAI models has profoundly impacted the field of artificial intelligence. Researchers are actively exploring the latest GPT (Generative Pre-trained Transformer) series, including GPT-3, GPT-3.5 Turbo, GPT-4o [168], GPT-4mini [170], and ChatGPT for malware generation, automated phishing attacks, malware detection, and analysis.

Li et al. [51] examined how ChatGPT enhances android malware detection by improving interpretability rather than making direct classification decisions. Traditional models like Drebin [171], MaMaDroid [172], and XM_{AL} [173] are effective but suffer from bias and lack of transparency, making it difficult for developers to trust their outputs. The study integrates ChatGPT into malware analysis workflows, using static and dynamic analysis techniques to extract app features and employing prompt engineering to generate detailed explanations of potential threats. While ChatGPT does not classify malware, it helps security analysts understand detection results, improving usability, trust, and decision-making in malware research. The findings highlight ChatGPT's value as an interpretability tool, complementing traditional mal-

ware detection systems by offering comprehensive explanations of security threats.

In addition to malware generation and detection, the GPT models automate phishing attacks. For example, Begou et al. [174] examined how ChatGPT, based on the GPT-3.5-turbo-16K model, can be exploited to develop and automate sophisticated phishing attacks. The study demonstrates how ChatGPT can assist in various aspects of phishing, including website cloning, credential theft integration, code obfuscation, domain registration, and phishing site deployment. The research highlights the vulnerabilities of AI-driven text models, revealing that despite safety measures, ChatGPT can be manipulated to generate and automate phishing websites that closely mimic legitimate ones. Similarly, in another study, Roy et al. [175] demonstrated how ChatGPT can be misused to generate sophisticated phishing emails, highlighting the potential for AI-driven tools to enhance the effectiveness of social engineering attacks.

Wang et al. [48] proposed an LLM-based workflow to analyze and detect anti-dynamic analysis techniques (TADA) in malware, which are designed to evade sandbox detection, debugging, and reverse engineering efforts. The authors incorporated the GPT-4 Turbo model to analyze the malware's assembly code features, string-based indicators, and API call sequences. GPT-4 was able to extract API call patterns, analyze assembly code features, and detect obfuscated strings used for sandbox evasion.

10.1.2. Llama

Llama 2, developed by Meta, is an open-weight Large Language Model (LLM) optimized for text generation, reasoning, and coding tasks. Due to its availability and ease of fine-tuning, it has been leveraged for cybersecurity research but also misused for malware generation, phishing automation, and exploit development by cybercriminals who modify its training data. Its open-source nature makes it a powerful yet dual-use AI system, requiring strict governance to prevent adversarial applications.

Simion et al. [176] evaluated the efficacy of open-source LLMs in detecting malware by analyzing sequences of API calls. The researchers assessed four LLMs-Llama2-13B, Mistral, Mixtral, and Mixtral-FP16-using a dataset comprising 20,000 malware samples and 20,000 benign files. The study concluded that when used without fine-tuning, these models

exhibited low accuracy and were unsuitable for real-time malware detection scenarios. This underscores the necessity for fine-tuning LLMs and integrating them with traditional security tools to enhance their effectiveness in identifying malicious software.

10.1.3. *Mixtral*

Mixtral is a Mixture-of-Experts (MoE) model developed by Mistral AI, combining multiple expert models to improve scalability and accuracy while maintaining lower computational costs. Hossain et al. [33] suggested an approach to detect malicious code in Java source files, incorporating the Mixtral model. Traditional signature-based and behavioral-based malware detection techniques struggle with zero-day attacks and obfuscation techniques, prompting the need for AI-driven solutions. The research introduces an LLM-powered detection system that analyzes Java code structures using graph-based methodologies.

10.1.4. *BERT*

BERT (Bidirectional Encoder Representations from Transformers): Created by Google, BERT is designed to understand the context of a word in search queries, improving the accuracy of search results.

Su et al. [56] inspected BERT's transformer-based architecture to enhance the detection of malicious URLs by capturing semantic relationships and contextual patterns in URL structures. Unlike traditional approaches that rely on lexical features or shallow machine learning models, BERT processes URLs using tokenization, self-attention, and deep contextual embeddings, allowing it to distinguish benign, phishing, malware, and defacement URLs with higher accuracy. Additionally, BERT demonstrates versatility by analyzing both raw URLs and structured feature representations (e.g., extracted API calls and request patterns) via multi-view input processing, making it applicable to various cybersecurity domains, including phishing detection, DNS-over-HTTPS attack identification, and IoT malware analysis. The results confirm BERT's performance in malicious URL detection, proving its effectiveness in real-time cybersecurity applications.

Similarly, in another study, Chang et al. [177] proposed a method that integrates the BERT model with Convolutional Neural Networks (CNN) to enhance malicious URL detection. This approach effectively captures both the contextual relevance and local features of URLs, leading to improved detection accuracy.

10.1.5. *Claude*

Claude [169], developed by Anthropic, is an AI-powered conversational model designed with a focus on safety, interpretability, and ethical AI alignment. Bard [178], rebranded as Gemini by Google DeepMind, is an advanced AI model designed for conversational AI, research assistance, and code generation. Roy et al. [57] inspected how commercially available LLMs, specifically ChatGPT (GPT-3.5 Turbo), GPT-4, Claude, and Bard, can be manipulated to generate phishing websites and emails. The study reveals that these LLMs can circumvent content moderation and produce highly convincing phishing attacks that mimic legitimate brands using recursive prompt engineering techniques. Attackers can automate and scale phishing content generation by reusing LLM-generated prompts, reducing the manual efforts required. Additionally, the authors disclose their findings to major LLM providers, leading to security updates. The research highlights the dual-use nature of LLMs, underscoring both their potential for malicious exploitation and the development of countermeasures to mitigate risks.

10.1.6. *Deepseek*

DeepSeek [179] is an open-weight LLM developed for advanced reasoning, code generation, and multilingual processing. Its high adaptability and fine-tuning capabilities have made it a valuable tool for cybersecurity research, including malware detection, exploit analysis, and penetration testing. The paper [180] investigates the security risks

associated with DeepSeek-R1, a Chain-of-Thought (CoT) reasoning-based LLM. The study specifically examines fine-tuning attacks, which manipulate the model's output by bypassing its safety alignment and ethical safeguards. The researchers conduct adversarial fine-tuning on DeepSeek-R1-Distill-Llama-8B, a distilled version of DeepSeek, using a dataset of harmful prompts. Experimental results reveal that fine-tuned DeepSeek exhibits a significant increase in generating harmful content, highlighting its vulnerability to malicious modifications. Compared to non-CoT models like Mistral-7B, DeepSeek's step-by-step reasoning process amplifies the generation of dangerous responses, making it particularly susceptible to security risks. The study underscores the critical need for stronger safeguards, regulatory oversight, and robust security mechanisms to prevent the exploitation of CoT-enabled LLMs for unethical or malicious purposes.

10.2. *Specialized offensive AI models and applications*

Unlike general-purpose LLMs, these models are specifically designed or fine-tuned for malicious purposes. They are developed without imposing any ethical restrictions and are used for phishing automation, malware obfuscation, exploit creation, and social engineering. These models are commonly found on dark web marketplaces and underground hacking forums, providing cybercriminals with advanced AI-driven tools. Furthermore, there are various malicious applications of LLMs are also discussed in the following subsections:

10.2.1. *Wormgpt*

WormGPT [181] is a malicious LLM developed by cyber criminals to facilitate fraudulent activities. Unlike legitimate LLMs designed for beneficial applications, WormGPT is tailored to assist in creating sophisticated phishing emails, crafting malicious code, and automating various cyberattacks [182]. Its primary usage revolves around enabling cybercriminals to execute more convincing and efficient fraudulent schemes utilizing advanced language generation capabilities.

10.2.2. *Fraudgpt*

FraudGPT is an advanced AI-driven tool that has been specifically designed to assist cybercriminals in conducting fraudulent activities at scale. FraudGPT is programmed without safety measures, making it a powerful tool for launching cyberattacks, including credential theft, phishing, malware development, and scam automation [182]. It enables malicious actors to generate convincing phishing emails, social engineering scripts, and exploit kits that evade traditional cybersecurity defenses [183].

10.2.3. *Evil-GPT*

Evil-GPT is a malicious AI chatbot that emerged on darknet forums, designed explicitly for cybercriminal activities [184]. Marketed as an alternative to tools like WormGPT, Evil-GPT is built on open-source LLMs and fine-tuned to help in generating malicious code, crafting sophisticated phishing emails, and developing hacking tools [185]. Its availability on underground markets lowers the barrier to entry for cybercriminals, enabling individuals with limited technical expertise to conduct complex cyberattacks.

10.2.4. *Darkgpt*

DarkGPT [186] represents a new category of LLMs specifically designed to function without conventional ethical constraints or content restrictions. Essentially, it serves as an "unfiltered" variant of ChatGPT, capable of generating illicit or sensitive human-like text [187]. It has been identified as one of several malicious AI chatbots marketed to cybercriminals, bypassing safety checks via "jailbreaking" to generate content that mainstream models would normally refuse [188]. DarkGPT is an advanced AI model capable of generating highly malicious code, excelling in developing sophisticated malware that not only compiles successfully but also evades antivirus detection. [189].

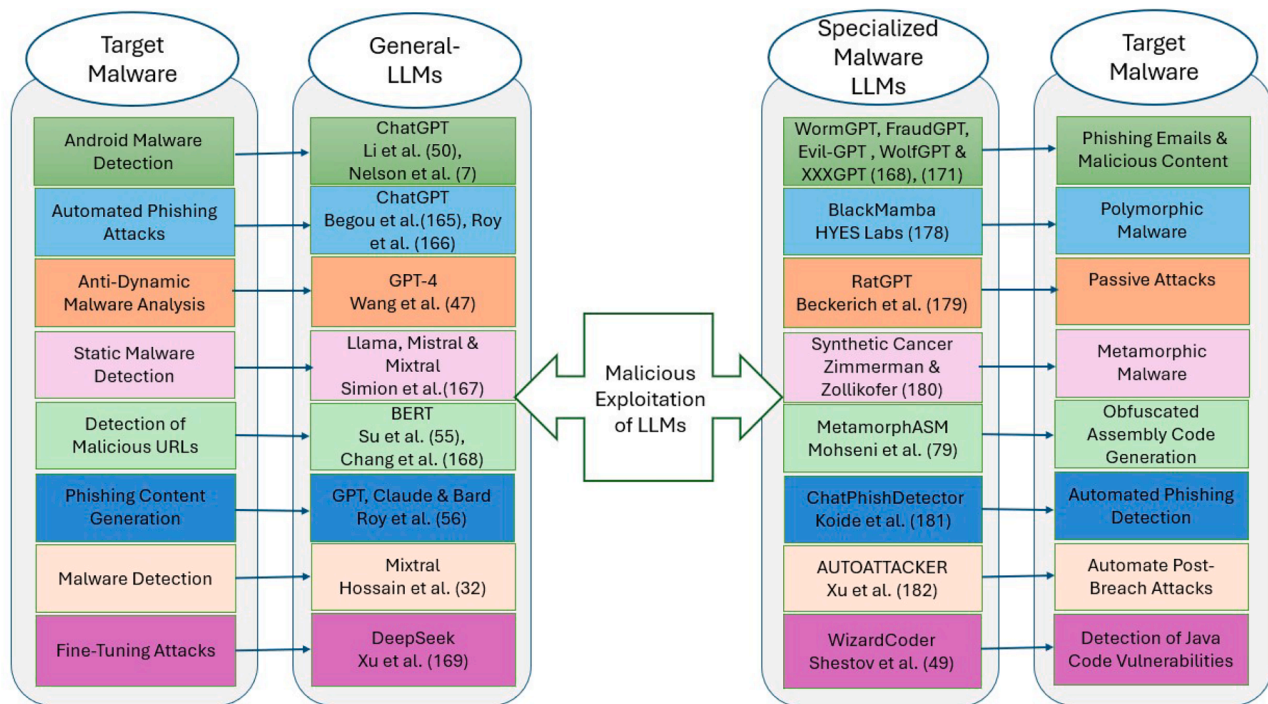


Fig. 12. Application of different LLMs for malware code analysis across various targets.

10.2.5. Wolf GPT

Wolf GPT is a malicious AI chatbot that has emerged on dark web forums. It is designed to assist cybercriminals in their illicit activities. Wolf GPT is built using open-source LLMs, it enables users to generate malicious code, craft sophisticated phishing emails, and develop hacking tools. Its availability lowers the barrier to entry for cybercriminals, allowing individuals with limited technical expertise to conduct complex cyberattacks.

10.2.6. XXX GPT

XXX GPT is another malicious AI chatbot that has been identified on hacker forums, offering cybercriminals a platform to automate and enhance their malicious endeavors. Similar to Wolf GPT, XXX GPT leverages advanced AI capabilities to facilitate the creation of malware, phishing schemes, and other cyber threats [190].

10.2.7. Blackmamba

BlackMamba is a proof-of-concept malware developed by HYAS Labs [191] that leverages artificial intelligence to dynamically generate polymorphic keylogging code at runtime, effectively evading traditional endpoint detection and response (EDR) systems. Upon execution, BlackMamba reaches out to OpenAI's API to synthesize malicious code, which is then executed within the benign program's context using Python's 'exec()' function, leaving no trace on the disk. This process allows BlackMamba to re-synthesize its keylogging capability with each execution, rendering the malicious component truly polymorphic. Captured keystrokes, including sensitive information such as usernames, passwords, and credit card numbers, are exfiltrated via legitimate communication channels like Microsoft Teams, further complicating detection efforts.

10.2.8. Ratgpt

Beckerich et al. [192] explore how LLMs, particularly ChatGPT, can be misused as intermediaries for malware attacks. The study demonstrates a proof-of-concept attack where an LLM is used as a proxy between an attacker and a victim, bypassing traditional cybersecurity measures. The researchers show how ChatGPT-generated payloads can be

used to weaponize seemingly harmless executables, enabling them to establish communication with a Command and Control (C2) server. By exploiting web-accessible plugins, the malware can dynamically generate IP addresses, execute commands, and exfiltrate data while remaining undetected. A key finding is that ChatGPT and similar LLMs can facilitate passive attacks, allowing malware to operate without direct interaction from the victim. The attack model relies on LLM-generated code for both payload delivery and remote execution, circumventing traditional malware detection methods since no hardcoded malicious code is stored on disk.

10.2.9. Synthetic cancer

Zimmerman and Zollikofer [193] present a prototype of metamorphic malware that utilizes LLMs for automatic code rewriting to evade signature-based detection. LLM alters malware's source code during replication, generating unique variants with each execution. This constant evolution allows it to bypass signature-based detection mechanisms employed by traditional antivirus programs. Additionally, the malware employs social engineering techniques, analyzing previous Email conversations to generate contextually relevant and persuasive responses. These responses include infected attachments, tricking recipients into opening them, and inadvertently spreading the malware further.

10.2.10. Metamorphasm

Mohseni et al. [79] examine how LLMs can generate obfuscated assembly code, challenging antivirus systems. They introduce the MetamorphASM benchmark with 328,200 obfuscated code samples, using techniques like dead code insertion and register substitution. The study evaluates LLMs like GPT-3.5, GPT-4, and others, using information-theoretic metrics and human reviews for assessment.

10.2.11. Chatphishdetector

Koide et al. [58], introduced ChatPhishDetector, a novel system that uses LLM to identify phishing websites. The system employs a web crawler to gather comprehensive website data, which is then transformed into prompts for LLM analysis. This approach enables the

detection of multilingual phishing sites by recognizing impersonated brands and social engineering tactics without the need for additional machine learning training.

10.2.12. AUTOATTACKER

The study [194] explored the performance of GPT-3.5-Turbo and GPT-4 within the AUTOATTACKER framework to automate post-breach cyberattacks such as privilege escalation, credential theft, and ransomware execution. It is an innovative LLM-based system to automate complex, human-like cyber-attacks, commonly known as “hands-on-keyboard” attacks. The system comprises four key components-Summarizer, Planner, Navigator, and Experience Manager that work together to generate precise attack commands tailored to various simulated environments. Extensive testing revealed that while models like GPT-3.5 and Llama2 variants showed limited effectiveness, GPT-4 demonstrated remarkable capabilities in conducting post-breach attacks with minimal human intervention.

10.2.13. WizardCoder

Shestov et al. [50] proposed WizardCoder, an advanced variant of StarCoder, which plays a crucial role in detecting vulnerabilities in Java source code. The model, with its 13-billion parameter architecture, is fine-tuned specifically for binary classification of vulnerable vs. non-vulnerable code functions. Compared to CodeBERT-based models, WizardCoder demonstrates higher accuracy and robustness in identifying security flaws. The paper highlights key improvements, such as adapting WizardCoder for classification tasks, handling imbalanced datasets, and optimizing training speed using batch-packing techniques. The fine-tuned WizardCoder outperforms ContraBERT, achieving better ROC AUC and F1 scores on both balanced and imbalanced datasets.

11. Malware datasets

Researchers need comprehensive and diverse malware datasets to analyze malware techniques and attack strategies effectively. One common approach to obtaining such datasets is through online platforms associated with anti-malware projects, including MalShare, Malware DB, and VirusShare. In addition, some organizations and research groups periodically release malware data sets to support cybersecurity research and mitigate the scarcity of publicly available sources.

Beyond direct collection, researchers have begun to explore the generation of synthetic malware datasets as an alternative. In particular, LLMs have emerged as a promising tool for generating malware code datasets, using the techniques discussed earlier in Section 9. LLMs can create synthetic malware samples, obfuscate malicious code, and generate variations of known threats, effectively expanding and diversifying existing malware datasets. Furthermore, data augmentation has become a powerful technique in enhancing malware detection models. For example, LLMs can simulate polymorphic and metamorphic malware, allowing machine learning-based detection systems to improve accuracy and robustness by training on adversarially generated malware variants. This approach has been instrumental in strengthening threat intelligence databases, improving malware classifiers, and enhancing cybersecurity defenses against rapidly evolving threats.

11.1. Ready-to-use malware datasets

Existing malware datasets are essential in cybersecurity research, providing structured collections of malicious software samples to aid in detection, analysis, and prevention. These datasets focus on various malware types and attack vectors, including Android malware (AndroZoo, DREBIN, KronoDroid), and Windows PE malware (BODMAS, SOREL-20M).

Additionally, repositories like VirusTotal and MalwareBazaar provide continuous updates of real-world malware samples, ensuring researchers have access to evolving threats. Public (open) datasets are

freely available or have minimal restrictions, enabling data-driven research, AI model training, and standardized benchmarking for malware detection. Restricted datasets require special permissions, passwords, or commercial access, offering high-quality, proprietary threat intelligence while maintaining security controls to prevent misuse. These datasets are crucial for identifying emerging threats, enhancing AI-driven security models, and enabling reliable cybersecurity solutions. Table 5 lists a mix of open and restricted malware data resources that can be used to obtain malware samples. Furthermore, a concise overview of some of the prominent datasets is provided below:

- **AndroZoo**[195] A comprehensive repository of Android malware samples, widely used for machine learning-based malware classification.
- **BODMAS**[196] A data set containing time-stamped malware samples and well-curated family information for research purposes.
- **DikeDataset** [197] This dataset comprises labeled samples, including benign and malicious portable executable (PE) and Object Linking and Embedding (OLE) files.
- **DREBIN**[171] A large-scale dataset for static and dynamic analysis of Android malware.
- **EMBER**[198] A dataset for portable executable (PE) malware analysis, widely used in machine learning-based malware feature engineering.
- **MalRadar**[199] A data set that focuses on Android malware, providing disarmed samples for research purposes.
- **Malware Datasets Top-1000 Imports**[200] It is a static malware analysis dataset that contains the top 1000 imported functions from Windows PE files.
- **MalwareBazaar**[102] A public threat intelligence sharing platform that offers access to diverse zero-day malware samples.
- **Malware Exploratory**[201] This dataset provides an in-depth analysis of malware samples, offering insights into malware behavior, patterns, and classifications to support cybersecurity research and threat intelligence.
- **Microsoft Malware Classification Challenge**[202] A dataset providing labeled Windows executable malware samples, often used for deep learning-based malware detection.
- **MOTIF**[203] It is the largest public malware dataset with ground truth family labels, facilitating malware classification and behavior analysis research.
- **SOREL-20M**[204] comprises metadata and features for approximately 20 million Windows Portable Executable (PE) files, including 10 million disarmed malware samples.
- **TheZoo**[205] is a live malware repository maintained for educational and research purposes.
- **VirusShare**[206] A well-known repository of multi-platform malware samples, but access is restricted to verified security researchers.
- **Vx-underground**[207] is one of the largest publicly available malware repositories, providing over 35 million malware samples, including source code and research materials.

11.2. Employing AI and LLMs for malware dataset augmentation

Data augmentation is a technique employed to artificially increase the size and diversity of a dataset by creating new samples through various transformations or modifications of existing data [218]. Dataset augmentation plays a critical role in enhancing malware detection within the NPM ecosystem. Yu et al. [219] utilized LLMs to translate malicious code snippets from multiple programming languages (e.g., Python, C, Go) into JavaScript, significantly expanding the available dataset. This cross-language translation ensures that Maltracker captures a wider range of attack patterns, allowing it to detect malware that traditional rule-based methods might overlook. By integrating these synthetically generated yet functionally equivalent malware variants into the dataset, the model improves its ability to recognize polymorphic

Table 5

Some of the impressive malware datasets for malware code analysis.

Malware Dataset (link)			Application	Target	Access Status	Content	Number of Files	
AndroZoo [208]			Malware Analysis	Android Malware	Requested Access	APK files, Metadata, VirusTotal Reports	25M + APKs	
BODMAS [209]			Malware Analysis	Windows PE Malware	Public Access	Labeled Windows PE malware dataset	60M + Samples	
Contagio Dump [210]	Malware		Malware Analysis	Various Malware Types	Password Protected	Clean and malicious files in various formats	16,800 clean and 11,960 malicious files	
DikeDataset [197]			Malware Analysis	PE and OLE Files	Public Access	Labeled benign and malicious PE and OLE samples	Not specified	
DREBIN [211]			Threat Detection	Mobile Threats	Requested Access	Android Malware Static Features	5560 Apps	
KronoDroid [212]			Android Malware Analysis	Android Malware	Public Access	Hybrid dataset with static and dynamic malware features	Samples from 2008 to 2020	
MalRadar [213]			Android Malware Analysis	Android Malware	Requested Access	Disarmed Android malware samples	4534 unique	Android malware samples
Top-1000 PE Imports [200]			Malware Analysis	Windows PE Imports	Registered Users	Top-1000 imported functions in PE files	Top-1000 Dataset	Imports
MalwareBazaar [102]			Threat Intelligence	Zero-Day Malware	Public Access	Malware Samples and Signatures	Ongoing Updates	
Malware-Exploratory [201]			Malware Analysis	General Malware	Open	Exploratory data analysis of malware samples	216,352 Samples	
Microsoft Classification [202]	Malware Challenge		Malware Analysis	Windows Executables	Public Access	PE Files with Labels	10M + Samples	
MOTIF [214]			Threat Detection	Malware Campaigns	Public Access	Attack Attribution and Threat Intelligence	3095 disarmed PE malware samples from 454 families	
SOREL-20M [215]			Malware Analysis	Windows PE Malware	Public Access	Feature-extracted malware samples	PE	20M + Samples
TaintBench [216]			Android Taint Analysis	Real-world Android Malware	Public Access	Benchmark suite for evaluating Android taint analysis tools	39 malware applications with 249 documented benchmark cases	
TheZoo [217]			Malware Analysis	Multi-platform Malware	Public Access	Executable malware samples for research	Multiple Families of Malware	
VirusShare [206]			Malware Analysis	Multi-platform Malware	Registered Users	Malware samples across different platforms	Ongoing Updates	
Vx-underground [207]			Malware Analysis	Multi-platform Malware	Public Access	Malware samples, source code, research papers	35M + Malware Samples	

and obfuscated threats. This highlights how AI-powered dataset expansion can significantly enhance the robustness of malware tracking and cybersecurity defenses in real-world software repositories like NPM.

Likewise, Palo Alto Networks Unit 42 researchers [220] utilized LLMs to iteratively transform malicious JavaScript code, generating numerous obfuscated variants that retained their original functionality but evaded detection by existing models. By integrating these LLM-generated samples into the training dataset, they effectively augmented the data, leading to improved model generalization. This approach demonstrates the efficacy of using LLMs for data augmentation to bolster the robustness of malware detection systems.

12. Discussions, limitations and suggestions

The field of LLMs for code analysis presents various challenges and open questions that warrant further research. Our study identifies key areas that remain insufficiently addressed and hold great potential for future advancements. In this section, we examine several critical challenges and identify gaps in existing work, outlining potential directions for future research.

12.1. Future work

The advancement of LLMs in malware analysis opens up numerous avenues for further exploration. In this section, we outline several potential directions for future work based on the current findings and limitations observed in our study. These proposed areas aim to address existing gaps, enhance the performance and reliability of LLM-based systems, and extend their applicability across diverse cybersecurity contexts. The following points highlight key research opportunities and practical developments that could shape the next generation of intelligent malware analysis tools.

- **Knowledge Graph Models with LLM for Malware** Knowledge graphs (KGs) are used to represent knowledge across different fields, helping to identify connections between entities. For example, by analyzing similarities between malware and attackers, KGs can reveal patterns. If multiple attackers use the same malware and techniques, they likely belong to the same organization. In a study by Hu et al. (2024) [221], they proposed an automated method for building a threat intelligence knowledge graph. This method uses GPT's few-shot learning ability to generate prompts, allowing for automatic data annotation and augmentation. This reduces the need for manual

work and creates datasets for training models. The Llama2-7B model was then fine-tuned to classify topics, extract entities and relationships, and identify TTPs (Tactics, Techniques, and Procedures) from threat intelligence reports, ultimately creating a useful knowledge graph.

- **Hybrid LLM-Program Analysis Frameworks for Malware Reverse Engineering.** Although LLMs show strong semantic reasoning capability, they lack precise control over low-level program semantics such as register state transitions and memory layouts. Hybrid frameworks that combine LLM reasoning with traditional reverse engineering tools (e.g., Ghidra, IDA Pro, angr, and symbolic execution engines) represent a promising research direction. Such systems could integrate static control-flow recovery, dynamic taint tracking, and LLM-based semantic summarization to produce interpretable malware behavior descriptions and accelerate analyst workflows in large-scale malware triage pipelines.
- **Chain-of-Thought Prompt methods for decompiling** To understand the application of Chain-of-Thought Prompt methods for decompiling, we first need to consider the role of CoT prompting in complex tasks. CoT prompting involves breaking down a problem step-by-step, allowing the model to reason through each component before concluding. When applied to decompiling, this method enables a more structured and logical approach to reverse engineering code or text, making it easier to understand the underlying processes and components. However, despite its potential, it remains an open challenge. The complexity of decompiling code or intricate structures, combined with the limitations of current models, means that achieving fully reliable and efficient decompiling using CoT prompting is still a work in progress.

For example, in [222], the authors propose applying CoT prompting, a recent advancement in NLP that guides LLMs through intermediate reasoning steps, to the task of decompiling WebAssembly binaries into high-level C code. This approach draws inspiration from human-like thinking processes to break down the complex task of decompilation into multiple phases, enabling the model to interpret, abstract, and translate binary code more effectively.

- **LLM for Malware Code and DFA DFDs** are helpful in malware analysis to track data flow and spot suspicious behavior in source code. Combining DFDs with NLP models like LLMs can improve malware analysis by using the strengths of both. DFDs track data flow and dependencies, while LLMs can find patterns, malicious intent, or hidden threats in text. There is little work on these methods; for example, in [223], the authors created a customizable framework for dataflow analysis using LLMs to analyze Java programs. They used the tree-sitter library to extract key information like parameters, return values, callers/callees, and sources/sinks. The framework was tested with four LLMs: GPT-3.5, GPT-4, Gemini-1.0, and Claude-3, and its performance was evaluated on real-world Android malware from the TaintBench Suite.
- **Knowledge-Enhanced Pre-trained Language Models KE-PLMs**, these models represent a significant advancement in NLP by integrating structured external knowledge into the pre-training phase of language models [224,225]. We can take advantage of these models for enhancing malware detection on source codes.

KE-PLMs incorporate external knowledge of various forms, including linguistic information, factual data, and domain-specific insights, to enrich the model's understanding and contextual awareness [226]. This approach enhances the model's ability to comprehend complex language structures and perform tasks requiring deep reasoning. KE-PLMs can perform better across a broad spectrum of NLP applications, contributing to more accurate and contextually relevant language processing outcomes by directly injecting external knowledge [225]. We believe that can find relationships and behaviors associated with malware activities on source codes, based on by incorporating structured external knowledge-such as informa-

tion from knowledge graphs and domain-specific data, KE-PLMs can better capture the intricate.

- **Large Concept Model (LCM) for malware code analysis** Imagine a cybersecurity firm receives a suspicious file with potential malware. Traditional models typically analyze individual code snippets, often overlooking sophisticated obfuscation techniques. These limitations stem from token-level processing. In [227], the authors introduced Large Concept Models (LCMs), which focus on concept-complete semantic units-rather than tokens. This method allows for improved long-context understanding, abstract reasoning, and more efficient computation, ultimately enhancing performance in tasks such as cross-lingual and multimodal applications.
- **Robust Evaluation Benchmarks for LLM-based Malware Analysis.** Existing evaluations remain fragmented across datasets, malware families, and task definitions. Future work should prioritize standardized benchmarking frameworks for LLM-based malware detection, family classification, deobfuscation, and binary-to-source matching, enabling fair cross-model performance comparison under reproducible experimental settings.

12.2. Limitations and challenges

Despite the promising capabilities of LLMs in malware analysis, several fundamental limitations and open challenges remain that hinder large-scale, real-world deployment.

- **Token-Length Constraints for Malware Decompilation.** LLMs operate under strict context window limitations, restricting their ability to process long assembly traces or complete malware binaries in a single pass. As a result, malware programs must be decomposed into smaller units such as individual functions or basic blocks, which may fragment long-range dependencies. Liu et al. [228] show that truncated context significantly degrades reasoning accuracy. Similarly, LLM4Decompile [78] is constrained by a maximum input length of 4096 tokens. Scaling context windows without sacrificing reasoning fidelity remains an open challenge.
- **Generalization to Novel Malware Families and Obfuscation Techniques.** LLMs rely heavily on the distribution of their training data. When encountering previously unseen malware families, zero-day exploits, or novel obfuscation strategies, performance can significantly degrade [229]. This limitation is particularly critical in dynamic threat environments where malware tactics rapidly evolve. Continual learning and online model adaptation remain underexplored for cybersecurity-scale LLM deployments [230].
- **Hallucination and Over-Confident Reasoning.** LLMs may generate plausible but incorrect explanations-a phenomenon known as hallucination-which poses risks in forensic malware analysis [231]. Over-confident but incorrect behavioral summaries may mislead analysts, especially in high-stakes incident response scenarios.
- **Adversarial Attacks Against LLM-Based Malware Detectors.** LLM-based security systems are themselves susceptible to adversarial prompt injection and evasion strategies [232,233]. Malware authors may craft inputs to manipulate LLM interpretations, leading to misclassification or information leakage. Robust adversarial training for security-oriented LLMs remains an open research challenge [234].
- **Explainability and Analyst Trust.** Although LLMs produce natural-language explanations, these explanations are not guaranteed to reflect true underlying program semantics [235]. Ensuring faithful and verifiable explanations is essential for analyst trust, regulatory compliance, and operational deployment [236].
- **Computational Cost and Deployment Constraints.** High-performance LLMs incur significant computational and energy costs, limiting their feasibility for real-time deployment in enterprise security systems, endpoint detection platforms, and large-scale malware scanning pipelines [237,238].

13. Conclusion

This paper provides a comprehensive overview of the potential of large language models (LLMs) in the field of malware code analysis. Our focus was primarily on transformer-based models and their capabilities in detecting, classifying, and understanding malicious code. These advanced natural language processing (NLP) models have shown great promise in addressing modern cybersecurity challenges, particularly in identifying and mitigating evolving malware threats.

The study discusses how LLMs can effectively analyze malicious code by leveraging their ability to recognize patterns, detect anomalies, and classify threats with high accuracy. By harnessing the power of deep learning and pre-trained models, LLMs have the potential to revolutionize the field of malware analysis, making it more efficient and scalable. Additionally, the study explores several publicly available datasets that are instrumental for malware research, particularly for tasks such as pre-training and fine-tuning LLMs. These datasets play a crucial role in static malware analysis, enabling researchers and cybersecurity professionals to train models on real-world malicious code samples.

Furthermore, the study examines how integrating LLMs into malware detection and mitigation strategies can enable more proactive and adaptive cybersecurity measures. By automating aspects of malware classification and code analysis, LLMs enhance detection speed, reduce human effort, and improve overall security posture. Their ability to adapt to new and evolving malware threats makes them a valuable tool in cybersecurity defense mechanisms.

Despite current advancements, LLMs are expected to hold significant potential for further enhancing malware analysis methodologies and strengthening cybersecurity defenses. As these models continue to evolve, they could play a pivotal role in shaping the future of threat intelligence, malware detection, and cybersecurity automation.

CRedit authorship contribution statement

Hamed Jelodar: Writing – original draft, Methodology, Conceptualization; **Samita Bai:** Writing – review & editing, Writing – original draft; **Parisa Hamedi:** Writing – review & editing, Writing – original draft; **Hesamodin Mohammadian:** Writing – review & editing, Writing – original draft; **Roozbeh Razavi-Far:** Writing – review & editing, Writing – original draft; **Ali Ghorbani:** Writing – review & editing, Writing – original draft.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Uncited References

Missing citation. [44]

References

- [1] Parla V, Zawadowskiy A, Anderson B, Latapie HM, Bessonov O, McGrew DA, et al. Llm technology for polymorphic generation of samples of malware for modeling, grouping, detonation and analysis. 2024. US Patent App. 18/360,676.
- [2] Jurecek M, Kozák M. Representation of PE files using LSTM networks. In: *ICISSP*. 2021, p. 516–25.
- [3] Kunwar P, Aryal K, Gupta M, Abdelsalam M, Bertino E. Sok: Leveraging transformers for malware analysis. *arXiv preprint arXiv:240517190*; 2024.
- [4] Tsfaty C, Fire M. Malicious source code detection using transformer. *arXiv preprint arXiv:220907957*; 2022.
- [5] Zhao W, Wu J, Meng Z. Appoet: large language model based android malware detection via multi-view prompt engineering. *Expert Syst Appl* 2025;262: 125546.
- [6] Fang R, Bindu R, Gupta A, Zhan Q, Kang D. Llm agents can autonomously hack websites. *arXiv preprint arXiv:240206664*; 2024.
- [7] Cani A, Gaudesi M, Sanchez E, Squillero G, Tonda A. Towards automated malware creation: code generation and code integration. In: *Proceedings of the 29th annual ACM symposium on applied computing*. 2014, p. 157–60.
- [8] Nelson J, Pavlidis M, Fish A, Kapetanakis S, Polatidis N. ChatGPT-driven machine learning code generation for android malware detection. *Comput J* 2024; bxae114.
- [9] Mohsin A, Janicke H, Wood A, Sarker IH, Maglaras L, Janjua N. Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse LLMs. *arXiv preprint arXiv:240612513*; 2024.
- [10] Chen J, Zhong Q, Wang Y, Ning K, Liu Y, Xu Z, et al. Rmcbench: benchmarking large language models' resistance to malicious code. In: *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*. 2024, p. 995–1006.
- [11] Ning K, Chen J, Zhong Q, Zhang T, Wang Y, Li W, et al. Mcgmark: An encodable and robust online watermark for llm-generated malicious code. *arXiv preprint arXiv:240801354*; 2024.
- [12] Xue D, Zhao G, Fan Z, Li W, Xu Y, Liu Z, et al. Poster: an exploration of large language models in malicious source code detection. In: *Proceedings of the 2024 on ACM SIGSAC conference on computer and communications security*. 2024, p. 4940–2.
- [13] Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:230812950*; 2023.
- [14] AI@Meta. Llama 3 model card 2024; https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- [15] Ye J, Chen X, Xu N, Zu C, Shao Z, Liu S, et al. A comprehensive capability analysis of gpt-3 and gpt-3.5 series models. *arXiv preprint arXiv:230310420*; 2023.
- [16] Anthropic. Introducing the next generation of claude 2024; <https://www.anthropic.com/news/claude-3-family>.
- [17] Guo D, Zhu Q, Yang D, Xie Z, Dong K, Zhang W, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:240114196*; 2024.
- [18] Qian X, Zheng X, He Y, Yang S, Cavallaro L. Lamd: Context-driven android malware detection and classification with llms. *arXiv preprint arXiv:250213055*; 2025.
- [19] AI@Meta. Jadx: dex to java decompiler 2024; <https://github.com/skylot/jadx>.
- [20] Omar M, Gouveia LB, Al-Karaki J, Mohammed D. Reverse-engineering malware. In: *Cybersecurity capabilities in developing nations and its impact on global security*. IGI Global Scientific Publishing; 2022, p. 194–217.
- [21] Eagle C. The IDA pro book. no starch press; 2011.
- [22] Eagle C, Nance K. The Ghidra Book: The Definitive Guide. no starch press; 2020.
- [23] Singh J, Singh J. Detection of malicious software by analyzing the behavioral artifacts using machine learning algorithms. *Inf Softw Technol* 2020;121: 106273.
- [24] Yao Y, Duan J, Xu K, Cai Y, Sun Z, Zhang Y. A survey on large language model (llm) security and privacy: the good, the bad, and the ugly. *High-Confid Comput* 2024; 100211.
- [25] Rong H, Duan Y, Zhang H, Wang X, Chen H, Duan S, et al. Disassembling obfuscated executables with LLM. *arXiv preprint arXiv:240708924*; 2024.
- [26] Sahoo P, Singh AK, Saha S, Jain V, Mondal S, Chadha A. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:240207927*; 2024.
- [27] McKinzie B, Gan Z, Fauconnier J-P, Dodge S, Zhang B, Dufter P, et al. Mm1: methods, analysis and insights from multimodal llm pre-training. In: *European conference on computer vision*. Springer; 2024, p. 304–23.
- [28] Rahali A, Akhloufi MA. Malbert: malware detection using bidirectional encoder representations from transformers. In: *2021 IEEE International conference on systems, man, and cybernetics (SMC)*. IEEE; 2021, p. 3226–31.
- [29] Setak M, Madani P. Fine-tuning LLMs for code mutation: a new era of cyber threats. In: *2024 IEEE 6th international conference on trust, privacy and security in intelligent systems, and applications (TPS-ISA)*. IEEE; 2024, p. 313–21.
- [30] Jin X, Barbieri F, Kennedy B, Davani AM, Neves L, Ren X. On transferability of bias mitigation effects in language model fine-tuning. *arXiv preprint arXiv:201012864*; 2020.
- [31] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [32] Khan M A-Z, Al-Karaki J, Omar M. Llms for malware detection: review, framework design, and countermeasure approaches. *Framework Des Countermeasure Approach* 2024.
- [33] Hossain AA, PK MK, Zhang J, Amsaad F. Malicious code detection using llm. In: *NAECON 2024-IEEE National aerospace and electronics conference*. IEEE; 2024, p. 414–16.
- [34] Zhang Y, Zhou X, Wen H, Niu W, Liu J, Wang H, et al. Tactics, techniques, and procedures (TTPs) in interpreted malware: A zero-shot generation with large language models. *arXiv preprint arXiv:240708532*; 2024.
- [35] Xu H, Wang S, Li N, Wang K, Zhao Y, Chen K, et al. Large language models for cyber security: A systematic literature review. *arXiv preprint arXiv:240504760*; 2024.
- [36] Al-Karaki J, Khan M A-Z, Omar M. Exploring llms for malware detection: Review, framework design, and countermeasure approaches. *arXiv preprint arXiv:240907587*; 2024.
- [37] Lu H, Peng H, Nan G, Cui J, Wang C, Jin W, et al. Malsight: Exploring malicious source code and benign pseudocode for iterative binary malware summarization.

- arXiv preprint arXiv:240618379; 2024.
- [38] Islam NT, Khoury J, Seong A, Bou-Harb E, Najafirad P. Enhancing source code security with LLMs: Demystifying the challenges and generating reliable repairs. arXiv preprint arXiv:240900571; 2024.
 - [39] HOANG XD, NINH T TT, PHAM HD. A NOVEL MODEL BASED ON DEEP TRANSFER LEARNING FOR DETECTING MALICIOUS JAVASCRIPT CODE. *J Theor Appl Inf Technol* 2024;102(18).
 - [40] Çetin O, Ekmekcioglu E, Arief B, Hernandez-Castro J. An empirical evaluation of large language models in static code analysis for PHP vulnerability detection. *J Univ Comput Sci* 2024;30(9):1163–83.
 - [41] Lin C-H, Pao H-K, Liao J-W. Efficient dynamic malware analysis using virtual time control mechanics. *Comput Secur* 2018;73:359–73.
 - [42] Fujii S, Yamagishi R. Feasibility study for supporting static malware analysis using LLM. arXiv preprint arXiv:241114905; 2024.
 - [43] Demirci D, Acarturk C, et al. Static malware detection using stacked bilstm and gpt-2. *IEEE Access* 2022;10:58488–502.
 - [44] Wang Y, Le H, Gohmare AD, Bui N DQ, Li J, Hoi S CH. Codet5+: Open code large language models for code understanding and generation. arXiv preprint arXiv:230507922; 2023.
 - [45] Walton BJ, Khatun ME, Ghawaly JM, Ali-Gombe A. Exploring large language models for semantic analysis and categorization of android malware. arXiv preprint arXiv:250104848; 2025.
 - [46] Or-Meir O, Nissim N, Elovici Y, Rokach L. Dynamic malware analysis in the modern era-a state of the art survey. *ACM Comput Surv (CSUR)* 2019;52(5):1–48.
 - [47] Huang Z, Pa Y MP, Yoshioka K. Exploring the impact of LLM assisted malware variants on anti-virus detection. In: 2024 IEEE Conference on dependable and secure computing (DSC). IEEE; 2024, p. 76–7.
 - [48] Wang H, Luo N, Liu P. Unmasking the shadows: Pinpoint the implementations of anti-dynamic analysis techniques in malware using llm. arXiv preprint arXiv:241105982; 2024.
 - [49] Devadiga D, Jin G, Potdar B, Koo H, Han A, Shringi A, et al. Gleam: gan and llm for evasive adversarial malware. In: 2023 14th international conference on information and communication technology convergence (ICTC). IEEE; 2023, p. 53–8.
 - [50] Shestov A, Cheshkov A, Levichev R, Mussabayev R, Zadorozhny P, Maslov E, et al. Finetuning large language models for vulnerability detection. arXiv preprint arXiv:240117010; 2024.
 - [51] Li Y, Fang S, Zhang T, Cai H. Enhancing android malware detection: The influence of chatGPT on decision-centric task. arXiv preprint arXiv:241004352; 2024.
 - [52] Wang Y, Fan M, Zhang X, Shi J, Qiu Z, Wang H, et al. Liredroid: llm-enhanced test case generation for static sensitive behavior replication. In: Proceedings of the 15th asia-Pacific symposium on internetware. 2024, p. 81–4.
 - [53] Pagano F, Verderame L, Russo E, Merlo A. Marvelhidedroid: reliable on-the-fly data anonymization based on android virtualization. *Comput Electr Eng* 2025;121:109882.
 - [54] Wang C, Zhang W, Su Z, Xu X, Xie X, Zhang X. Llm4df: analyzing dataflow in code with large language models. *Adv Neural Inf Process Syst* 2025;37:131545–74.
 - [55] Roy SS, Nilizadeh S. Phishlang: A lightweight, client-side phishing detection framework using mobileBERT for real-time, explainable threat mitigation. arXiv preprint arXiv:240805667; 2024.
 - [56] Su M-Y, Su K-L. Bert-based approaches to identifying malicious urls. *Sensors* 2023;23(20):8499.
 - [57] Roy SS, Thota P, Naragam KV, Nilizadeh S. From chatbots to phishbots?: phishing scam generation in commercial large language models. In: 2024 IEEE Symposium on security and privacy (SP). IEEE Computer Society; 2024, p. 221.
 - [58] Koide T, Nakano H, Chiba D. Chatphishdetector: detecting phishing sites using large language models. *IEEE Access* 2024.
 - [59] Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, et al. Mixtral of experts. arXiv preprint arXiv:240104088; 2024.
 - [60] Tóth R, Bisztray T, Erdődi L. Llm in web development: evaluating llm-generated php code unveiling vulnerabilities and limitations. In: International conference on computer safety, reliability, and security. Springer; 2024, p. 425–37.
 - [61] Wong G-W, Huang Y-T, Guo Y-R, Sun Y, Chen MC. Attention-based API locating for malware techniques. *IEEE Trans Inf Forensics Secur* 2023.
 - [62] Patsakis C, et al. Assessing large language models in reverse engineering tasks. arXiv preprint arXiv:240200000; 2024.
 - [63] Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, et al. A conversational paradigm for program synthesis. arXiv preprint arXiv:220313474; 2022.
 - [64] Adamec M, Turčaník M. Development of malware using large language models. In: 2024 New trends in signal processing (NTSP). IEEE; 2024, p. 1–5.
 - [65] Bao H, Li W, Chen H, Miao H, Wang Q, Tang Z, et al. Stories behind decisions: towards interpretable malware family classification with hierarchical attention. *Comput Secur* 2024;144:103943.
 - [66] Sun B, Li Q, Guo Y, Wen Q, Lin X, Liu W. Malware family classification method based on static feature extraction. In: 2017 3rd IEEE international conference on computer and communications (ICCC). IEEE; 2017, p. 507–13.
 - [67] Zhiwu XU, Ren K, Song F. Android malware family classification and characterization using CFG and DFG. In: 2019 International symposium on theoretical aspects of software engineering (TASE). IEEE; 2019, p. 49–56.
 - [68] Su Z, Xu X, Huang Z, Zhang Z, Ye Y, Huang J, et al. Codeart: better code models by attention regularization when symbols are lacking. *Proc ACM Softw Eng* 2024;1(FSE):562–85.
 - [69] Christodorescu M, Jha S. Static analysis of executables to detect malicious patterns. In: 12th USENIX security symposium (USENIX security 03). 2003.
 - [70] Samhi J, Li L, Bissyandé TF, Klein J. Difuzer: uncovering suspicious hidden sensitive operations in android apps. In: Proceedings of the 44th international conference on software engineering. 2022, p. 723–35.
 - [71] Mohamed N. A comprehensive review of natural language processing techniques for malware detection. In: 2024 15th international conference on computing communication and networking technologies (ICCCNT). IEEE; 2024, p. 1–7.
 - [72] Medvet E, Mercaldo F. Exploring the usage of topic modeling for android malware static analysis. In: 2016 11th international conference on availability, reliability and security (ARES). IEEE; 2016, p. 609–17.
 - [73] Xu W, Hu W, Wu F, Sengamedu S. DetiME: Diffusion-enhanced topic modeling using encoder-decoder based LLM. arXiv preprint arXiv:231015296; 2023.
 - [74] Li Z, Dutta S, Naik M. Llm-assisted static analysis for detecting security vulnerabilities. arXiv preprint arXiv:240517238; 2024.
 - [75] Huang Y, Wang R, Zheng W, Zhou Z, Wu S, Ke S, et al. Spiderscan: practical detection of malicious NPM packages based on graph-based behavior modeling and matching. In: Proceedings of the 39th IEEE/ACM international conference on automated software engineering. 2024, p. 1146–58.
 - [76] Bhardwaj V, Kukreja V, Sharma C, Kansal I, Popali R. Reverse engineering-a method for analyzing malicious code behavior. In: 2021 International conference on advances in computing, communication, and control (ICAC3). IEEE; 2021, p. 1–5.
 - [77] Botacin M, Galante L, de Geus P, Grégio A. Revenge is a dish served cold: debug-oriented malware decompilation and reassembly. In: Proceedings of the 3rd reversing and offensive-oriented trends symposium. 2019, p. 1–12.
 - [78] Tan H, Luo Q, Li J, Zhang Y. Llm4decompile: Decompiling binary code with large language models. arXiv preprint arXiv:240305286; 2024.
 - [79] Mohseni S, Mohammadi S, Tilwani D, Saxena Y, Ndwlua G, Vema S, et al. Can LLMs obfuscate code? a systematic analysis of large language models into assembly code obfuscation. arXiv preprint arXiv:241216135; 2024.
 - [80] Muralidharan T, Cohen A, Gerson N, Nissim N. File packing from the malware perspective: techniques, analysis approaches, and directions for enhancements. *ACM Comput Surv* 2022;55(5):1–45.
 - [81] Chukkol A HA, Luo S, Sharif K, Haruna Y, Abdullahi MM. Vulcatch: Enhancing binary vulnerability detection through codet5 decompilation and KAN advanced feature extraction. arXiv preprint arXiv:240807181; 2024.
 - [82] Lin Y, Wan C, Fang Y, Gu X. Codecipher: learning to obfuscate source code against LLMs 2024;2410.05797.
 - [83] Uchendu I, et al. Llm-assisted deobfuscation of malware. *Comput Secur* 2024.
 - [84] Royal P, et al. Polypack: an automated online packing service for malware. In: ACSAC. 2006.
 - [85] Collberg C, Thomborson C, Low D. A taxonomy of obfuscating transformations. In: Department of computer science, university of auckland, technical report. 1997.
 - [86] Udupa S, Debray S, Madou M. Automatic deobfuscation of obfuscated binaries. In: ACM CCS. 2005.
 - [87] Banescu S, et al. Breaking opaque predicates. In: IEEE Security and privacy workshops. 2016.
 - [88] Xu G, et al. Extracting API call sequences for malware classification. *IEEE Trans Dependable Secure Comput* 2022.
 - [89] You I, Yim K. Malware obfuscation techniques: a brief survey. 2010 Int Conf Broadband, Wireless Comput Commun Appl 2010.
 - [90] Egele M, Scholte T, Kirda E, Kruegel C. A survey on automated dynamic malware analysis techniques and tools. *ACM Comput Surv* 2012;44(2).
 - [91] Guo S, et al. X-Force: multi-layer malware unpacking. In: USENIX Security. 2019.
 - [92] Hofmann J, et al. Dynamic software unpacking using taint analysis. In: IEEE S&P workshops. 2020.
 - [93] Massarelli L, et al. Safe: self-attentive function embeddings for binary similarity. In: NDSS. 2019.
 - [94] Li Y, et al. Palmtree: learning an assembly-level semantic representation using contextualized embedding. In: USENIX Security. 2021.
 - [95] Phan M, et al. Ir2vec: learning program representations at the intermediate representation level. In: ICSE. 2020.
 - [96] Xu L, et al. Gemini: learning binary code similarity with graph embeddings. In: IEEE Symposium on security and privacy. 2017.
 - [97] Pepe D, et al. Trec: learning execution semantics from program traces. In: NeurIPS. 2019.
 - [98] Jiang L, An J, Huang H, Tang Q, Nie S, Wu S, et al. Binaryai: binary software composition analysis via intelligent binary source code matching. In: Proceedings of the IEEE/ACM 46th international conference on software engineering. 2024, p. 1–13.
 - [99] Kim S, et al. Binsearch: Binary function matching with generative language models. arXiv preprint arXiv:240400000; 2024.
 - [100] Poudyal S, Subedi KP, Dasgupta D. A framework for analyzing ransomware using machine learning. In: 2018 IEEE Symposium series on computational intelligence (SSCI). IEEE; 2018, p. 1692–9.
 - [101] VirusTotal. Virustotal. 2023. Accessed on February 17, 2025; <https://www.virustotal.com/gui/home/upload>.
 - [102] Malwarebazaar. Accessed on February 17, 2025; <https://bazaar.abuse.ch/>.
 - [103] Anyrun. Accessed on February 27, 2025; <https://any.run/>.
 - [104] Madan S, Sofat S, Bansal D. Tools and techniques for collection and analysis of internet-of-things malware: a systematic state-of-art review. *J King Saud Univ - Comput Inf Sci* 2022;34(10, Part B):9867–88. <https://doi.org/10.1016/j.jksuci.2021.12.016>
 - [105] Sun N, Ding M, Jiang J, Xu W, Mo X, Tai Y, et al. Cyber threat intelligence mining for proactive cybersecurity defense: a survey and new perspectives. *IEEE Commun Surv Tutor* 2023;25(3):1748–74. <https://doi.org/10.1109/COMST.2023.3273282>
 - [106] Tounsi W, Rais H. A survey on technical threat intelligence in the age of sophisticated cyber attacks. *Comput Secur* 2018;72:212–33. <https://doi.org/10.1016/j.cose.2018.07.001>

- cose.2017.09.001
- [107] Sun Z, Rao Z, Chen J, Xu R, He D, Yang H, et al. An opcode sequences analysis method for unknown malware detection. In: Proceedings of the 2019 2nd international conference on geoinformatics and data analysis. 2019, p. 15–19.
- [108] Mowri RA, Siddula M, Roy K. Application of explainable machine learning in detecting and classifying ransomware families based on API call analysis. *arXiv preprint arXiv:221011235*; 2022.
- [109] Wei X, Li C, Lv Q, Li N, Sun D, Wang Y. Mitigating the impact of malware evolution on API sequence-based windows malware detector. *arXiv preprint arXiv:240801661*; 2024.
- [110] Gajdošík A, Kotuliak I. Machine learning and transformers for malware analysis: overview. 2024 New Trends in Signal Process (NTSP) 2024; 1–7.
- [111] Kakisim AG, Gulmez S, Sogukpinar I. Sequential opcode embedding-based malware detection method. *Comput Electric Eng* 2022;98:107703.
- [112] Khan KN, Ullah N, Ali S, Khan MS, Nauman M, Ghani A. Op2vec: an opcode embedding technique and dataset design for end-to-end detection of android malware. *Secur Commun Netw* 2022;2022(1):3710968.
- [113] Zhang J, Qin Z, Yin H, Ou L, Zhang K. A feature-hybrid malware variants detection using CNN based opcode embedding and BPNN based API embedding. *Comput Secur* 2019;84:376–92.
- [114] Hu Q, Wang W, Song H, Guo S, Zhang J, Zhang S. Asdroid: resisting evolving android malware with api clusters derived from source code. *IEEE Trans Inf Forensics Secur* 2025;20:1822–35.
- [115] Yousefi-Azar M, Hamey L, Varadarajan V, Chen S. Learning latent byte-level feature representation for malware detection. In: Neural information processing: 25th international conference, ICONIP 2018, siem reap, cambodia, december 13–16, 2018, proceedings, part IV 25. Springer; 2018, p. 568–78.
- [116] Abusnaina A, Wang Y, Arora S, Wang K, Christodorescu M, Mohaisen D. Burning the adversarial bridges: Robust windows malware detection against binary-level mutations. *arXiv preprint arXiv:231003285*; 2023.
- [117] Korczynski D, Yin H. Capturing malware propagations with code injections and code-reuse attacks. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. 2017, p. 1691–708.
- [118] Mosli R, Slota TJ, Pan Y. Creating adversarial malware examples through guided metamorphic changes. In: 2021 IEEE International symposium on technologies for homeland security (HST). IEEE; 2021, p. 1–7.
- [119] Ding W, Abdel-Basset M, Ali AM, Moustafa N. Large language models for cyber resilience: a comprehensive review, challenges, and future perspectives. *Appl Soft Comput* 2025;170:112663. <https://doi.org/10.1016/j.asoc.2024.112663>
- [120] Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, et al. Large language models for software engineering: a systematic literature review. *ACM Trans Software Eng Method* 2024;33(8):1–79.
- [121] Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q. Software testing with large language models: survey, landscape, and vision. *IEEE Trans Software Eng* 2024.
- [122] Yu J, Liang P, Fu Y, Tahir A, Shahin M, Wang C, et al. An insight into security code review with LLMs: Capabilities, obstacles and influential factors. *arXiv preprint arXiv:240116310*; 2024.
- [123] Thapa C, Jang SI, Ahmed ME, Camtepe S, Pieprzyk J, Nepal S. Transformer-based language models for software vulnerability detection. In: Proceedings of the 38th annual computer security applications conference. 2022, p. 481–96.
- [124] Szabó Z, Bilicik V. A new approach to web application security: utilizing gpt language models for source code inspection. *Future Internet* 2023;15(10):326.
- [125] Noever D. Can large language models find and fix vulnerable software? *arXiv preprint arXiv:230810345*; 2023.
- [126] Gao Z, Wang H, Zhou Y, Zhu W, Zhang C. How far have we gone in vulnerability detection using large language models. *arXiv preprint arXiv:231112420*; 2023.
- [127] Luo Z, Xu C, Zhao P, Sun Q, Geng X, Hu W, et al. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:230608568*; 2023.
- [128] Bhandari G, Naseer A, Moonen L. Cvefixes: automated collection of vulnerabilities and their fixes from open-source software. In: Proceedings of the 17th international conference on predictive models and data analytics in software engineering. 2021, p. 30–9.
- [129] Ponta SE, Plate H, Sabetta A, Bezzi M, Dangremont C. A manually-curated dataset of fixes to vulnerabilities of open-source software. In: 2019 IEEE/ACM 16th international conference on mining software repositories (MSR). IEEE; 2019, p. 383–7.
- [130] Wang S, Zhang Y, Bao L, Xia X, Wu M. Vcmatch: a ranking-based approach for automatic security patches localization for OSS vulnerabilities. In: 2022 IEEE international conference on software analysis, evolution and reengineering (SANER). IEEE; 2022, p. 589–600.
- [131] Chan A, Kharkar A, Moghaddam RZ, Mohylevskyy Y, Helyar A, Kamal E, et al. Transformer-based vulnerability detection in code at edittime: Zero-shot, few-shot, or fine-tuning? *arXiv preprint arXiv:230601754*; 2023.
- [132] Kim M, Xin Q, Sinha S, Orso A. Automated test generation for rest apis: no time to rest yet. In: Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis. 2022, p. 289–301.
- [133] Kim M, Stennett T, Shah D, Sinha S, Orso A. Leveraging large language models to improve REST API testing. In: Proceedings of the 2024 ACM/IEEE 44th international conference on software engineering: new ideas and emerging results. 2024, p. 37–41.
- [134] Pereira A, Lima B, Faria JP. Apitestgenie: Automated api test generation through generative ai. *arXiv preprint arXiv:240903838*; 2024.
- [135] Kim M, Sinha S, Orso A. Llamarestest: Effective REST API testing with small language models. *arXiv preprint arXiv:250108598*; 2025.
- [136] Elkhail AA, Cerny T. On relating code smells to security vulnerabilities. In: 2019 IEEE 5th intl conference on big data security on cloud (bigdatasecurity), IEEE intl conference on high performance and smart computing (HPSC) and IEEE intl conference on intelligent data and security (IDS). IEEE; 2019, p. 7–12.
- [137] Kambhampati V, Mohammed NH, Fard AM. Characterizing javascript security code smells. *arXiv preprint arXiv:241119358*; 2024.
- [138] Zhang B, Liang P, Zhou X, Zhou X, Lo D, Feng Q, et al. A comprehensive evaluation of parameter-efficient fine-tuning on method-level code smell detection. *arXiv preprint arXiv:241213801*; 2024.
- [139] Wu D, Mu F, Shi L, Guo Z, Liu K, Zhuang W, et al. ISMELL: assembling LLMs with expert toolsets for code smell detection and refactoring. In: Proceedings of the 39th IEEE/ACM international conference on automated software engineering. 2024, p. 1345–57.
- [140] Mesbah D, El Madhoun N, Al Agha K, Chalouati H. Leveraging prompt-based large language models for code smell detection: a comparative study on the MLCQ dataset. In: The 13-th international conference on emerging internet, data & web technologies (EIDWT-2025). 2025.
- [141] Madeyski L, Lewowski T. Mlcq: industry-relevant code smell data set. In: Proceedings of the 24th international conference on evaluation and assessment in software engineering. 2020, p. 342–7.
- [142] Rasheed Z, Sami MA, Waseem M, Kemell K-K, Wang X, Nguyen A, et al. Ai-powered code review with llms: Early results. *arXiv preprint arXiv:240418496*; 2024.
- [143] Lucas K, Gheyri R, Soares E, Ribeiro M, Machado I. Evaluating large language models in detecting test smells. *arXiv preprint arXiv:240719261*; 2024.
- [144] Mousavi Z, Islam C, Moore K, Abuadba A, Babar MA. An investigation into misuse of java security apis by large language models. In: Proceedings of the 19th ACM asia conference on computer and communications security. 2024, p. 1299–315.
- [145] Altorfer F CS, Kelly MJ, Avrumova F, Rohatgi V, Zhu J, Bono CM, et al. The double-edged sword of generative AI: surpassing an expert or a deceptive "false friend"? *Spine J* 2025; <https://doi.org/10.1016/j.spinee.2025.02.010>
- [146] Botacin M. Gphtests-3: Is automatic malware generation a threat?. In: 2023 IEEE security and privacy workshops (SPW). 2023.
- [147] Pa Pa YM, Tanizaki S, Kou T, Van Eeten M, Yoshioka K, Matsumoto T. An attacker's dream? exploring the capabilities of chatgpt for developing malware. In: Proceedings of the 16th cyber security experimentation and test workshop. 2023, p. 10–18.
- [148] Shen X, Chen Z, Backes M, Shen Y, Zhang Y. "Do anything now": characterizing and evaluating in-the-wild jailbreak prompts on large language models. In: Proceedings of the 2024 ACM SIGSAC conference on computer and communications security. 2024, p. 1671–85.
- [149] Haldar R, Hockenmaier J. Analyzing the performance of large language models on code summarization. *arXiv preprint arXiv:240408018*; 2024.
- [150] Liu C, Zhang SD, Ibrahimzada AR, Jabbarvand R. Codemind: A framework to challenge large language models for code reasoning. 2024. 2402.09664; <https://arxiv.org/abs/2402.09664>.
- [151] Gupta M, Akiri K, Aryal K, Parker E, Praharaj L. From chatgpt to threatgpt: impact of generative ai in cybersecurity and privacy. *IEEE Access* 2023.
- [152] Alahmed Y, Abadla R, Al Ansari MJ. Exploring the potential implications of AI-generated content in social engineering attacks. In: 2024 International conference on multimedia computing, networking and applications (MCNA). IEEE; 2024, p. 64–73.
- [153] Charan PV, Chunduri H, Anand PM, Shukla SK. From text to mitre techniques: Exploring the malicious use of large language models for generating cyber attack payloads. *arXiv preprint arXiv:230515336*; 2023.
- [154] Altman S. OpenAI. 2022. Accessed on February 13, 2025; <https://chat.openai.com>.
- [155] Google bard. 2023. Accessed on February 13, 2025; <https://bard.google.com/>.
- [156] Lavina D. How to jailbreak chatgpt. 2023. <https://watcher.guru/news/how-to-jailbreak-chatgpt>.
- [157] Lee K. ChatGPT.DAN. 2023. https://github.com/0xk1h0/ChatGPT_DAN.
- [158] White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, et al. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:230211382*; 2023.
- [159] Li H, Guo D, Fan W, Xu M, Huang J, Meng F, et al. Multi-step jailbreaking privacy attacks on chatgpt. *arXiv preprint arXiv:230405197*; 2023.
- [160] Huang Y, Gupta S, Xia M, Li K, Chen D. Catastrophic jailbreak of open-source llms via exploiting generation. *arXiv preprint arXiv:231006987*; 2023.
- [161] OpenAI. Gpt-3.5. 2023. Accessed on February 13, 2025; <https://platform.openai.com/docs/models/gpt-3-5>.
- [162] Gravitas S. Auto-GPT: An autonomous GPT-4 experiment. 2023. <https://github.com/Significant-Gravitas/AutoGPT>.
- [163] Monje A, Monje A, Hallman RA, Cybenko G. Being a bad influence on the kids: malware generation in less than five minutes using chatgpt. Publisher: Unpublished 2023.
- [164] Botacin M. Malware samples and analysis logs. 2020. Accessed on February 11, 2025; <https://github.com/marcusbotaicin/malware-data>.
- [165] The red report 2023. 2023. <https://www.piscusecurity.com/resource/blog/the-red-report-2023-top-ten-attack-techniques>.
- [166] Liu Y, Deng G, Xu Z, Li Y, Zheng Y, Zhang Y, et al. Jailbreaking chatgpt via prompt engineering: An empirical study. *arXiv preprint arXiv:230513860*; 2023.
- [167] Michael L. Pe malware machine learning dataset. Jun. 08, 2021. Accessed on 04 March 2025; <https://practicalsecurityanalytics.com/pemalware-machine-learning-dataset/>.
- [168] OpenAI. GPT-4. 2024. Accessed on March 03, 2025; <https://platform.openai.com/docs/models/gpt-4o>.
- [169] Anthropic introducing claude. Accessed on 19 March 2025; <https://www.anthropic.com/index/introducing-claude>.
- [170] OpenAI. GPT-4. 2024. Accessed on March 03, 2025; <https://platform.openai.com/docs/models/gpt-4o-mini>.
- [171] Arp D, Spreitzenbarth M, Hubner M, Gascon H, Rieck K, Siemens C. Drebin: effective and explainable detection of android malware in your pocket. In: Ndss; vol. 14.

- 2014, p. 23–6.
- [172] Onwuzurike L, Mariconti E, Andriotis P, Cristofaro ED, Ross G, Stringhini G. *Madroid: detecting android malware by building markov chains of behavioral models (extended version)*. ACM Trans Privacy Secur (TOPS) 2019;22(2):1–34.
- [173] Wu B, Chen S, Gao C, Fan L, Liu Y, Wen W, et al. Why an android app is classified as malware: toward malware classification interpretation. ACM Trans Softw Eng Methodol (TOSEM) 2021;30(2):1–29.
- [174] Begou N, Vinoy J, Duda A, Korczynski M. Exploring the dark side of AI: Advanced phishing attack design and deployment using chatGPT. 2023. <https://arxiv.org/abs/2309.10463>.
- [175] Roy SS, Naragam KV, Nilizadeh S. Generating phishing attacks using chatgpt. arXiv preprint arXiv:230505133; 2023.
- [176] Simion C-A, Balan G, Gavriluț DT. Benchmarking out of the box open-source LLMs for malware detection based on API calls sequences. In: International conference on intelligent data engineering and automated learning. Springer; 2024, p. 133–42.
- [177] Chang W, Du F, Wang Y. Research on malicious URL detection technology based on BERT model. In: 2021 IEEE 9th international conference on information, communication and networks (ICICN). IEEE; 2021, p. 340–5.
- [178] Hsiao, Collins S, E. Try bard and share your feedback, blog google 2023; <https://blog.google/technology/ai/try-bard>.
- [179] Bi X, Chen D, Chen G, Chen S, Dai D, Deng C, et al. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:240102954; 2024.
- [180] Xu Z, Gardiner J, Belguith S. The dark deep side of deepseek: Fine-tuning attacks against the safety alignment of cot-enabled models. arXiv preprint arXiv:250201225; 2025.
- [181] Mohamed Firdhous MF, Elbreiki W, Abdullahi I, Sudantha BH, Budiarto R. WormGPT: a large language model chatbot for criminals. In: 2023 24th international arab conference on information technology (ACIT). 2023, p. 1–6. <https://doi.org/10.1109/ACIT58888.2023.10453752>
- [182] Falade PV. Decoding the threat landscape: Chatgpt, fraudgpt, and wormgpt in social engineering attacks. arXiv preprint arXiv:231005595; 2023.
- [183] Krishnan R. FraudGPT: The villain avatar of chatGPT. 2023. <https://netenrich.com/blog/fraudgpt-the-villain-avatar-of-chatgpt>.
- [184] After wolfGPT and wormGPT, evil-GPT surfaces on hacker forum. 2023. Accessed on March 18, 2025; <https://theycyberexpress.com/wolfgpt-wormgpt-evil-gpt-surface-hacker-forum/>.
- [185] Hollingworth D. Evil-GPT is the latest malicious AI chatbot to hit the darknet. 2023. <https://www.cyberdaily.au/tech/9420-evil-gpt-is-the-latest-malicious-ai-chatbot-to-hit-the-darknet>.
- [186] <https://flowgpt.com/p/darkgpt-official-edition>.
- [187] Rutherford A. 2024. <https://powerbrainai.com/dark-gpt/>.
- [188] Anonymous. Incident number 736: underground market for LLMs powers malware and phishing scams. AI Incident Database 2023; Retrieved March 2025 from <https://incidentdatabase.ai/cite/736>.
- [189] Lin Z, Cui J, Liao X, Wang X. Malla: demystifying real-world large language model integrated malicious services. In: 33rd USENIX security symposium (USENIX security 24). 2024, p. 4693–710.
- [190] Hackers released new black hat AI tools XXXGPT and wolf GPT. 2023. Accessed on March 18, 2025; <https://presale1.com/home/f/hackers-released-new-black-hat-ai-tools-xxxgpt-and-wolf-gpt>.
- [191] Blackmamba: Using AI to generate polymorphic malware. 2023. Accessed on March 13, 2025; <https://www.hyas.com/blog/blackmamba-using-ai-to-generate-polymorphic-malware>.
- [192] Beckerich M, Plein L, Coronado S. Ratgpt: Turning online llms into proxies for malware attacks. arXiv preprint arXiv:230809183; 2023.
- [193] Zimmerman B, Zollikofer D. Synthetic cancer-augmenting worms with LLMs. arXiv preprint arXiv:240619570; 2024.
- [194] Xu J, Stokes JW, McDonald G, Bai X, Marshall D, Wang S, et al. Autoattacker: A large language model guided system to implement automatic cyber-attacks. arXiv preprint arXiv:240301038; 2024.
- [195] Allix K, Bissyandé TF, Klein J, Le Traon Y. Androzoo: collecting millions of android apps for the research community. In: Proceedings of the 13th international conference on mining software repositories. MSR '16. ISBN 978-1-4503-4186-8; 2016, p. 468–71. <http://doi.acm.org/10.1145/2901739.2903508>.
- [196] Yang L, Ciptadi A, Laziuk I, Ahmadzadeh A, Wang G. Bodmas: an open dataset for learning based temporal analysis of pe malware. In: 2021 IEEE Security and privacy workshops (SPW). IEEE; 2021, p. 78–84.
- [197] Dike dataset. 2022. Accessed on March 13, 2025; <https://github.com/iosifache/DikeDataset>.
- [198] Anderson HS, Roth P. Ember: an open dataset for training static pe malware machine learning models. arXiv preprint arXiv:180404637; 2018.
- [199] Wang L, Wang H, He R, Tao R, Meng G, Luo X, et al. Malradar: demystifying android malware in the new era. Proc ACM Measur Anal Comput Syst 2022;6(2):1–27.
- [200] AngeloOliveira. Malware analysis datasets: Top-1000 PE imports. 2019. Accessed on March 13, 2025; <https://www.kaggle.com/datasets/ang3oliveira/malware-analysis-datasets-top1000-pe-imports>.
- [201] Malware exploratory. 2020. Accessed on March 13, 2025; <https://www.kaggle.com/code/phbarros/malware-exploratory>.
- [202] Ronen R, Radu M, Feuerstein C, Yom-Tov E, Ahmadi M. Microsoft malware classification challenge. arXiv preprint arXiv:180210135; 2018.
- [203] Joyce RJ, Amlani D, Nicholas C, Raff E. Motif: a malware reference dataset with ground truth family labels. Comput Secur 2023;124:102921.
- [204] Harang R, Rudd EM. Sorel-20m: A large scale benchmark dataset for malicious pe detection. arXiv preprint arXiv:201207634; 2020.
- [205] thezoo. Accessed on February 17, 2025; <https://github.com/ytisf/theZoo>.
- [206] Virusshare. Accessed on March 13, 2025; <https://virusshare.com/>.
- [207] Vx-underground (malware source code). Accessed on March 13, 2025; <https://github.com/vxunderground/MalwareSourceCode>.
- [208] Allix K, Bissyandé TF, Klein J, Le Traon Y. Androzoo. 2016. Accessed on March 13, 2025; <https://androzoo.uni.lu/>.
- [209] Yang L, Ciptadi A, Laziuk I, Ahmadzadeh A, Wang G. Bodmas malware dataset. 2022. Accessed on March 06, 2025; <https://whyisyoung.github.io/BODMAS/>.
- [210] Contagio malware dump. Accessed on March 13, 2025; <https://github.com/zollak/contagiodump>.
- [211] Arp D, Spreitzenbarth M, Hubner M, Gascon H, Rieck K. The drebin dataset. 2014. Accessed on March 13, 2025; <https://drebin.mlsec.org/>.
- [212] Manzanara AG. The kronodroid dataset. Accessed on March 13, 2025; <https://github.com/aleguma/kronodroid>.
- [213] Oliveira A. Malware analysis datasets: Top-1000 PE imports. 2019. <https://doi.org/10.21227/004e-v304>
- [214] Joyce RJ, Amlani D, Nicholas C, Raff E. Motif dataset. 2022. Accessed on March 13, 2025; <http://github.com/boozallen/MOTIF>.
- [215] Harang R, Rudd EM. Sophos-reversinglabs 20 million dataset. 2020. Accessed on March 13, 2025; <https://github.com/sophos/SOREL-20M>.
- [216] Luo L, Pauck F, Piskachev G, Benz M, Pashchenko I, Mory M, et al. Taintbench suite. 2022. <https://taintbench.github.io/taintbenchSuite/>.
- [217] Thezoo dataset. 2014. Accessed on March 13, 2025; <https://github.com/ytisf/theZoo>.
- [218] DeVries T, Taylor GW. Dataset augmentation in feature space. arXiv preprint arXiv:170205538; 2017.
- [219] Yu Z, Wen M, Guo X, Jin H. Maltracker: a fine-grained npm malware tracker copilot by llm-enhanced dataset. In: Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis. 2024, p. 1759–71.
- [220] Li Hu, Lucas, S Sarker, B Melicher, A Starov, W Wang, N Mohamed, and T Li . Now you see me, now you don't: Using LLMs to obfuscate malicious javascript. 2024. <https://unit42.paloaltonetworks.com/using-llms-obfuscate-malicious-javascript/>.
- [221] Hu Y, Zou F, Han J, Sun X, Wang Y. Llm-tikg: threat intelligence knowledge graph construction utilizing large language model. Comput Secur 2024;145:103999.
- [222] Fang W, Zhou Z, He J, Wang W. Stacksight: Unveiling webassembly through large language models and neurosymbolic chain-of-thought decompilation. arXiv preprint arXiv:240604568; 2024.
- [223] Wang C, Zhang W, Su Z, Xu X, Xie X, Zhang X. Llm4fa: analyzing dataflow in code with large language models. Adv Neural Inf Process Syst 2024;37:131545–74.
- [224] Yang J, Hu X, Xiao G, Shen Y. A survey of knowledge enhanced pre-trained language models. ACM Trans Asian Low-Resour Lang Inf Process 2024.
- [225] Hu L, Liu Z, Zhao Z, Hou L, Nie L, Li J. A survey of knowledge enhanced pre-trained language models. IEEE Trans Knowl Data Eng 2023;36(4):1413–30.
- [226] Zhen C, Shang Y, Liu X, Li Y, Chen Y, Zhang D. A survey on knowledge-enhanced pre-trained language models. 2022. 2212.13428; <https://arxiv.org/abs/2212.13428>.
- [227] Barrault L, Duquenne P-A, Elbayad M, Kozhevnikov A, Alastruay B, Andrews P, et al. Large concept models: Language modeling in a sentence representation space. arXiv preprint arXiv:241208821; 2024.
- [228] Liu M, Wang J, Lin T, Ma Q, Fang Z, Wu Y. An empirical study of the code generation of safety-critical software using llms. Appl Sci 2024;14(3):1046.
- [229] Lu C, Carlo G, Van Roy B. Out-of-distribution generalization in machine learning. IEEE Trans Pattern Anal Mach Intell 2022.
- [230] Parisi GI, Kemker R, Part JL, Kanan C, Wermter S. Continual lifelong learning with neural networks: a review. Neural Netw 2019;113:54–71.
- [231] Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, et al. Survey of hallucination in natural language generation. ACM Comput Surv 2023;55(12).
- [232] Liu Y, Deng S, Wang H, Wang H. Prompt injection attacks and defenses in large language models. arXiv preprint arXiv:230605499; 2023.
- [233] Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:230715043; 2023.
- [234] Madry A, Makelov A, Schmidt L, Tsipras D, Vladu A. Towards deep learning models resistant to adversarial attacks. In: International conference on learning representations (ICLR). 2018.
- [235] Rudin C. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Mach Intell 2019;1:206–15.
- [236] Doshi-Velez F, Kim B. Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:170208608; 2017.
- [237] Strubell E, Ganesh A, McCallum A. Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th annual meeting of the association for computational linguistics (ACL). 2019.
- [238] Patterson D, Gonzalez J, Le Q, Liang C, Munguia L, Rothchild D, et al. Carbon emissions and large neural network training. arXiv preprint arXiv:210410350; 2021.
- [239] Ganapathyappan K, Yadav A. Optimized deep learning technique for the effective detection of windows pe malware. In Cyberwarfare, security and space computing. Cham: Springer Nature Switzerland, 2025, pp. 359–370.
- [240] Sharma R, Ghosh M, Singh A. Ensemble-based transfer learning for enhanced ransomware detection using pe header. In 2024 3Rd edition of IEEE delhi section flagship conference (DELCON). 2024, pp. 1–6.
- [241] Hashwanth S, Kirthica S. Effective ransomware detection method using pe header and yara rules. In Proceedings of international conference on network security and blockchain technology. Singapore: Springer Nature Singapore, 2024, pp. 185–194.
- [242] Ngwobia SC, Ralescu A, Kapp D, Tebede T. Detection of malicious pe files using synthesized dna artifacts. Comput Secur, 134, 103457, 2023.

- [243] Matin IMM. Ransomware extraction using static portable executable (pe) feature-based approach. In 6Th international conference of computer and informatics engineering (IC2ie). 2023, pp. 70–74.
- [244] Tyagi S, Baghela A, Dar KM, Patel A, Kothari S, Bhosale S. Malware detection in pe files using machine learning. In OPJU International technology conference on emerging technologies for sustainable development (OTCON). 2023, pp. 1–6.
- [245] Kattamuri SJ, Penmatsa RKV, Chakravarty S, Madabathula VSP. Swarm optimization and machine learning applied to pe malware detection towards cyber threat intelligence. *Electronics (Basel)*, 12, 2, 2023.
- [246] ElNeel L, Copiaco A, Obaid W, Mukhtar H. Comparison of feature extraction and classification techniques of pe malware. In 5Th international conference on signal processing and information security (ICSPIS). 2022, pp. 26–31.
- [247] Fonseca FH, Ferracci S, Palmaro F, Iocchi L, Nardi D, Franchina L. Static analysis of pe files using neural network techniques for a pocket tool. In International conference on electrical, computer, communications and mechatronics engineering (ICECCME). 2021, pp. 01–06.
- [248] Manavi F, Hamzeh A. Static detection of ransomware using lstm network and pe header. In 26Th international computer conference, computer society of iran (CS-ICC). 2021, pp. 1–5.